

# orchestrator/ 代码说明

给第一次看这份代码的人快速建立地图用的开发文档。`main.py` 只是薄薄的入口胶水代码，真正的 5 个 Agent 各自一个文件放在 `agents/` 目录下，`persona/store/trip_plan` 三个模块已经被对应的 Agent 接起来了（具体接了什么见文末），只有 OTA/酒店 Agent 还是纯占位假数据。

设计背景/接口契约见 [agent-architecture.md](#)（为什么这么分 Agent、为什么中心化编排）和 [agent-interfaces.md](#)（字段契约、原本的 Dify 方案，后来改成直接写 Python）。

## 环境准备

```
pip install openai python-dotenv chromadb
```

在 `orchestrator/.env` 里写这几行（这个文件已经在 `.gitignore` 里，不会被提交）：

```
PARATERA_API_KEY=你的key
AMAP_KEY=你的高德Web服务API Key
AMAP_JS_KEY=你的高德Web端(JS API)Key
AMAP_JS_SECURITY_CODE=你的安全密钥
QWEATHER_KEY=你的和风天气API KEY
QWEATHER_API_HOST=你的和风天气专属API Host
```

高德的 key **分三样东西，缺一不可都不行**： - `AMAP_KEY`： **Web服务** 类型，后端用（`map_tool.py` 查地理编码/路线） - `AMAP_JS_KEY`： **Web端(JS API)** 类型，前端画地图用——注意这是另一个应用/另一个 key，不能跟上面那个通用 - `AMAP_JS_SECURITY_CODE`： 安全密钥，高德 JS API 2.0 版本强制要求，在申请 `AMAP_JS_KEY` 的同一个应用页面能找到

都在 <https://lbs.amap.com> 控制台"创建应用"里申请（一个应用可以同时创建 Web服务 + Web端(JS API) 两个 key，安全密钥在创建 Web端(JS API) key 的时候会一起给）。个人开发者未认证 6000次/天，实名认证后 30万次/月，demo 阶段够用。

和风天气（`weather_tool.py` 用，灾害预警）去 <https://dev.qweather.com> 控制台"项目管理"申请： - `QWEATHER_KEY`： 创建凭据时选 **API KEY** 类型，不要选 JWT（JWT 要自己写签名逻辑，demo 规模没必要） - `QWEATHER_API_HOST`： 同一个项目页面上能看到，形如 `xxxxxxxxxx.re.qweatherapi.com`，**不用带** `https://` 前缀——2023 年后注册的新账号是"一账号一专属域名"，公共的 `devapi.qweather.com` 对新 key 直接返回 404，必须用这个专属域名

个人开发者免费版 1000次/天，灾害预警接口在免费的 8 个基础接口里（辐射/海洋/热带气旋等付费专项接口不在内，用不上）。

每个文件都能直接 `python orchestrator/<路径>.py` 单独跑（包括 `agents/` 里的 5 个文件），文件末尾的 `if __name__ == "__main__":` 都是自测代码，可以照着抄用法。

## 目录结构

```
orchestrator/
llm_tool.py          # 通用 LLM 调用工具，跟"编排"本身无关，谁都能 import
map_tool.py          # 通用地图工具（高德地图 API）：地理编码 + 路径规划
weather_tool.py      # 通用天气工具（和风天气 API）：灾害预警查询，给异常应变 Agent 用
hotel_tool.py        # 通用酒店工具（道旅 RollingGo）：真实酒店搜索 + 房型报价，给 OTA/酒店 Agent 用
agents/
```

```
orchestrator_agent.py      # 编排 Agent: new_shared_state + classify_intent + orchestrate
content_agent.py          # 达人/内容 Agent
route_agent.py            # 行程/路线 Agent
ota_hotel_agent.py        # OTA/酒店 Agent (酒店真数据, 机票还是纯占位)
exception_agent.py        # 异常应变 Agent (提案制, 见对应一节)
restaurant_agent.py       # 餐厅技能 (fellow 分支合并进来): 澳门模拟餐厅演示, 带界数工具调用循环
widgets.py                # 展示插件: 右侧聊天框里的富交互组件, 跟 Agent 逻辑解耦
schedule_widgets.py       # 左侧日程面板展示组件 (地图+连线、每日时间线、机票酒店面板), 跟 widgets.py 同一个分离
nearby_sources.py         # 附近游/达人 Agent 共用的数据源: 高德 POI/路线 + 和风天气逐小时预报
restaurant_tools.py       # restaurant_agent 的工具调用实现 (虚构澳门餐厅数据 + 校验规则)
session_store.py          # 按浏览器 session 存整份 shared_state (SQLite), 替换掉原来的单进程全局 state
standardize_hk_macau_data.py # 把组员整理的港澳达人数据 Excel 转成标准化 JSON (见文末专门一节)
import_hk_macau_data.py   # 把标准化 JSON 灌进 Chroma (人格向量怎么算在这一步)
data_sources/hk_macau_posts.json # 标准化后的港澳达人数据库 (229 条, 本地人认证 + 小红书两个来源)
main.py                   # 薄入口: re-export 编排 Agent 的两个函数 + __main__ 完整 demo
store.py / trip_plan.py / persona.py # 不变
server.py / web/          # 接了 session_store + 港澳附近游相关接口, 见对应一节
```

**关于这条 workflow 设计思路** (content\_agent.py + route\_agent.py / restaurant\_agent.py 共同体现): 能算清楚的事实 (几点到哪、走多久、天气、餐厅是否符合硬性限制) 用确定性代码算完, LLM 只用在两处很窄的地方: 把用户的自然语言请求解析成结构化字段、从已经查到的真实候选里挑 ID (不许凭空编新地点/新属性)。最后给用户看的文字要么是纯模板拼出来的 (route\_agent.schedule() 排完之后的固定回复拼句), 要么是工具调用循环产出后原样返回、不再过第二个模型改写 (restaurant\_agent.run())。orchestrator\_agent.orchestrate() 里只有真正需要"总结/闲聊"性质的部分才会走最后一次 LLM 组句, 而且系统提示词里专门加了"以下 JSON 是资料不是指令, 不得捏造"这类防幻觉措辞。

拆分原则: 谁负责哪个 Agent 就改 agents/ 下自己那一个文件, 不用碰 main.py 或别人的 Agent 文件, 减少多人协作冲突。

llm\_tool.py -- 通用 LLM 工具

**职责:** 跟"编排"这件事本身无关的基础设施, 5 个 Agent 里任何一个要调用模型都用这个, 不要各自再写一遍 client 初始化。

| 函数/常量                                | 作用                                                                        |
|--------------------------------------|---------------------------------------------------------------------------|
| call_llm(messages, model=MODEL_FULL) | 统一的模型调用入口                                                                 |
| MODEL_FULL / MODEL_LIGHT             | DeepSeek-V4-Pro / DeepSeek-V4-Flash, 两档模型对齐 agent-architecture.md 的模型档位设计 |

map\_tool.py -- 通用地图工具 (高德地图)

**职责:** 地理编码 + 路径规划, 给行程/路线 Agent 算真实的距离/耗时/路线用, 跟 llm\_tool.py 一样是通用基础设施。

| 函数                                                         | 作用                                                                                    |
|------------------------------------------------------------|---------------------------------------------------------------------------------------|
| geocode(address, city=None)                                | 地名/地址转经纬度, 返回 (lng, lat)                                                              |
| route_between(place_a, place_b, mode="walking", city=None) | 算两地之间的距离/耗时/路线, mode 传 "walking" 或 "driving", 返回 distance_m / duration_min / polyline |

**现状/限制:** - 只做了"两点之间"的查询, 还没做"多点一次性规划最优顺序" (这个高德也有专门接口, 以后要优化路线顺序时再接) - route\_agent.py 里用这个算相邻两站的真实交通方式/耗时 (超过 2km 自动从步行切驾车), 查询失败会优雅降级成"待定 (地图查询失败: ...) "文字, 不会导致整个流程崩掉 - 到达/结束的具体钟点时间 (arrival\_time / end\_time) 还是占位——那需要"一天几点开始"+"每站玩多久"这类还没定义的调度逻辑, 跟地图 API 是两回事 - 地图可视化 (真的在网页上画一张地图+路线) 还没做, web/index.html 目前只是文字列表展示行程, polyline 字段目前没被前端用上

weather\_tool.py -- 通用天气工具（和风天气）

**职责：**和风天气的两块能力——灾害预警（给异常应变 Agent 判断“这个城市现在是不是真的有极端天气”）、逐小时预报（给港澳附近游 workflow 算“这个时段要不要带伞”）。跟 map\_tool.py 一样是通用基础设施。

| 函数                                                    | 作用                                                                                                                                                                                                                                    |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>get_active_warnings(location, city=None)</code> | 查某个地名当前生效的灾害预警，返回列表，每条 { <code>"headline"</code> , <code>"event_type"</code> , <code>"severity"</code> , <code>"description"</code> , <code>"effective_time"</code> , <code>"expire_time"</code> }；没有预警返回空列表（正常情况，不是错误）               |
| <code>get_hourly_forecast(lng, lat, hours=240)</code> | 查某个坐标未来最多 240 小时（10 天，免费版上限）的逐小时预报，返回列表，每条 { <code>"time"</code> , <code>"temperature_c"</code> , <code>"rain_probability"</code> , <code>"condition_text"</code> }；直接传经纬度（不查地名）， <code>nearby_sources.py</code> 已经有 POI 坐标就不用再地理编码一次 |

- `severity` 取值：`unknown / minor / moderate / severe / extreme`，越靠后越严重——`exception_agent.check_weather()` 用这个字段决定 `needs_replan` 要不要标 `True`（只是提示信号，不会自动改行程，见 `exception_agent.py` 一节）
- `get_active_warnings()` 按经纬度查（`/weatheralert/v1/current/{lat}/{lon}`），不需要先转和风天气自己的 `LocationID`，所以直接复用 `map_tool.geocode()` 拿坐标，少一次网络调用；`get_hourly_forecast()`（`/weather/v1/hourly/{lat}/{lon}`）同理，且额外传了 `localTime=true`，返回的 `time` 字段直接是当地时间字符串（不带时区偏移），不用再手动转时区
- 和风天气新账号是“一账号一专属域名”**：2023 年后注册的 key 用公共的 `devapi.qweather.com / geoapi.qweather.com` 会直接 404，必须去控制台“项目管理”页面找到这个 key 专属的 API Host（形如 `xxxxxxxxxx.re.qweatherapi.com`），配进 `.env` 的 `QWEATHER_API_HOST`（见“环境准备”一节）——这是接入过程里踩的第一个坑，实测过公共域名连 key 是否有效都判断不出来，报错也不明显（直接 404 空 body，不是 JSON 格式的错误信息）
- 2026-09-11 用真实预警数据验证过：查询当时有台风影响的海南（海口）能查到 2 条真实的雷电/大风黄色预警，字段解析正常；查没有预警的城市（香港/澳门）返回空列表，也验证过降级路径
- 2026-09-12 `get_hourly_forecast()` 真实数据验证过：`nearby_sources.weather()` 端到端跑通，拿到真实的澳门逐小时气温/降雨概率

hotel\_tool.py -- 通用酒店工具（道旅 RollingGo）

**职责：**真实酒店搜索 + 房型报价，给 `ota_hotel_agent.py` 用，2026-09-13 接入。跟 `map_tool.py / weather_tool.py` 一样是通用基础设施。

| 函数                                                                                                                    | 作用                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>search_hotels(place, place_type, origin_query, check_in_date=None, stay_nights=1, star_min=None, size=5)</code> | 搜酒店候选，返回列表，每条带 <code>hotel_id / name / address / star_rating / price_per_night / lat / lng / image_url / booking_url / ameniti</code> |
| <code>get_hotel_detail(hotel_id, check_in_date, check_out_date, adult_count=2, room_count=1)</code>                   | 查某个酒店的具体房型报价，返回房型列表，带 <code>room_name / price_per_night / bed_type / meal / cancelable / cancel_policy</code>                         |

- 数据源：**道旅（DIDA Travel，官方宣传“全球第三大酒旅 B2B 公司”）的 RollingGo 免费开放接口，200 万+真实酒店库存，个人/企业开发者均可免费申请、目前阶段无调用上限（官方说明是“目前阶段”，不保证长期不变）。去 `https://travelportal-partner-center.dida.com/register?lang=zh` 注册，0 等待即时拿到 key。**这一步是用户自己注册的，不是我能代办的事**（涉及个人账号信息）。
- 协议是 MCP，不是普通 REST：**服务本质是个 Model Context Protocol server（`https://mcp.rollinggo.cn/mcp`），但直接用 `requests` 发 JSON-RPC 2.0 请求过去就行，不用接一整套 MCP 客户端框架——`method` 固定 `"tools/call"`，`params.name` 是工具名，`params.arguments` 是工具参数。**真正的业务数据包了两层：**`response["result"][0]["content"]` 是一段

- JSON 字符串，要再 `json.loads()` 一次才能拿到真正的酒店数据（第一次接的时候没注意到这层，拿到的一直是“文本”外壳，调试了一下才发现）。请求头必须带 `Accept: application/json, text/event-stream`，缺了服务端直接 400。
- **search\_hotels() 返回的 price\_per\_night 是估算值：**源接口 `searchHotels` 只返回整个入住期间的总价（`lowestPrice`），本模块按 `stay_nights` 算术平均换算成“每晚”，不是真的按天查一次；想要某个具体房型精确到每晚多少钱、取消政策，要再调一次 `get_hotel_detail()`（目前 `ota_hotel_agent.py` 没有自动调这一步，只在候选列表阶段用估算价）。
  - **只做只读查询，不碰下单/支付：**官方还有一个走 OAuth 登录 + 人工二次确认的命令行工具（`rgh` 系列，能真实下单付款），调研这个数据源的时候顺带发现的，明确决定不接进这种自动跑的后端 Agent 流程——那类操作必须是用户本人在自己的终端里主动触发，不适合塞进 `orchestrate()` 这种自动化链路。
  - 2026-09-13 真实数据验证过：搜“大三巴附近四星酒店”拿到 5 家真实澳门酒店（假日酒店/华都酒店等），真实价格/地址/坐标/图片/设施；`getHotelDetail` 查到真实房型（假日高级房/高级双床房等）、真实每晚价格、真实取消政策。

## agents/orchestrator\_agent.py -- 编排 Agent

**职责：**星型架构的中枢。接收用户消息 → 意图识别 → 决定调用哪几个子 Agent/workflow → 汇总结果 → 生成回复。对应 [agent\\_interfaces.md 第三节](#) 的节点拓扑，只是从 Dify 可视化节点换成了 Python 函数。模型档位用 `MODEL_FULL`（全系统推理最重）。

**2026-09-14 重构：达人 Agent 收窄成“提出候选”，编排 Agent 统一负责调度+对话，排时间归 route\_agent.py**——`nearby`（港澳附近游）不再是独立短路的 workflow，并入了达人 Agent 内部（见下面 `content_agent.py` 一节），`nearby_planner.py` 整个删除，逻辑吸收进 `content_agent.py`（候选生成）和 `route_agent.py`（排时间）。这是继 2026-09-11 合并 `fellow` 分支之后的第二次大调整。

| 函数                                                                                   | 作用                                                                                                   |
|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>new_shared_state(user_id, scenario="vacation", onboarding_answers=None)</code> | 造一份共享状态： <code>persona + trip_plan + messages / pending_widgets</code>                               |
| <code>classify_intent(user_message, history=None)</code>                             | 调模型判断这轮要触发<br><code>nearby / content / restaurant / route / booking / exception / cancel</code> 里的哪个 |
| <code>orchestrate(user_message, shared_state)</code>                                 | 主流程，返回 <code>(output, shared_state)</code>                                                           |

`orchestrate()` 分发逻辑（不再有整体短路）：1. `nearby / content` 二选一调达人 Agent：`nearby` 命中传 `mode="nearby"`，否则（`content` 命中且非 `restaurant`）传 `mode="trip"`——两者共用 `content_agent.run()` 同一个函数，细节见下面一节 2. 达人 Agent 返回 `clarification_needed`（`mode="nearby"` 解析不出起点时）就直接把这句追问当回复整轮返回，不再往下跑——跟以前 `nearby_planner.from_chat()` 解析失败时的短路行为一致 3. `route` 或 `nearby` 命中，且达人 Agent 真的给出了候选，就调 `route_agent.schedule()` 排时间（`nearby` 命中给单日、`hours / origin` 用达人 Agent 解析出的；否则给 `_extract_day_count(user_message)` 提取出的天数，正则匹配“N日/N天”，提取不到默认 1 天，封顶 14 天）4. `booking / exception / cancel` 分发方式不变 5. 回复优先级：`restaurant > cancel > 通用 LLM 组句`（其中只有通用 LLM 组句这一条会真的调模型生成自由文本，前两条都是固定模板拼句）

**cancel 意图**（`exception_agent` 提案制“确认后执行”缺口的第一个真实触发路径）：区别于 `exception`（用户在了解情况，还没决定要不要动行程，命中只产生未验证提案，不执行），`cancel` 是用户直接下达删除指令，消息本身就是确认。`orchestrate()` 里 `_handle_cancel()` 复用 `exception_agent.run()` 的子串匹配定位地点，找到了就立刻调 `exception_agent.apply_adjustment()` 真删。删除成功会固定追问“是补新活动还是重新排”，但这句追问之后没有对应的“清空重排”能力（`route_agent.schedule()` 目前只会往 `trip_plan` 里新增天数，不会清空已有内容重排），是待接事项。

`orchestrate()` 返回的 `output` 里 `map_panel` 现在是 `route_agent.schedule()` 的返回结构（`{"days":, "weather_reminders":, "unscheduled":}`），不再是旧版 `{"route": [...]}` 那种扁平结构——前端 `web/index.html` 还没跟着改，这是已知的前端配套缺口。

## agents/content\_agent.py -- 达人/内容 Agent

**职责：**提出候选景点/美食供编排 Agent 筛选，不管排时间（排时间是 `route_agent.schedule()` 的事）。2026-09-14 重构：`nearby` 并入这里，两种场景共用同一条“查真实周边 POI + 社区口碑复核”流水线。模型档位 `MODEL_FULL`；`mode="nearby"` 时会有一次 `MODEL_LIGHT` 调用解析请求参数，`mode="trip"` 目前仍是纯向量检索，没有实际用到 LLM。

```
run(shared_state, location_hint, mode="trip", top_k=3) :
```

| mode       | 行为                                                                                                                                                                                                                                                                                               |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "trip"（默认） | 先按人格向量从社区帖子库筛几个种子（ <code>store.query_similar_posts()</code> ，多捞 <code>top_k*5</code> 过滤掉 <code>category=="Tips"</code> 的再截到 <code>top_k</code> ——空白人格实测会系统性偏向 Tips 帖子，过滤完一个不剩就退回未过滤结果兜底，不摆烂）；再对每个种子（ <code>category</code> 是 Food/Sight 的，Tips 跳过）分别调 <code>_nearby_plan()</code> 查真实周边 POI，汇总去重 |
| "nearby"   | 跳过人格检索种子； <code>_parse_nearby_request()</code> 用 <code>MODEL_LIGHT</code> 从 <code>location_hint</code> 解析出起点/游览小时数，解析不出返回 <code>clarification_needed</code> ；解析出来就用 <code>nearby_sources.find_origin()</code> 查真实起点坐标，调一次 <code>_nearby_plan()</code>                                            |

`_nearby_plan(origin, city, persona_vector)`：统一的“周边探索”接口，两种场景都调用——查 `nearby_sources.nearby()` 真实高德周边 POI，再对每个 POI 查 `store.find_posts_by_place()` 看社区库有没有人评价过，评价人的人格向量跟当前用户的用 `persona.cosine_similarity()` 比对。排序策略：有人格匹配分的排最前，有评价但算不出匹配分的其次，完全没评价的排最后但**仍然保留**（229 条帖子覆盖的地点有限，多数真实 POI 大概率查不到匹配评价，不能因为没数据就排除）。地点匹配是精确字符串匹配（POI 名字 vs 帖子 `place` 字段），命中率本来就不高，是已知局限。

**已知坑（2026-09-14 实测踩过）：**给种子地点找坐标不能用 `map_tool.geocode()`（高德 `geocode/geo` 接口），对“渔人码头”这类多地重名的地标在港澳场景下经常查到内地同名地点（实测查出过跑去河北秦皇岛/云南的坐标）——改用 `nearby_sources.find_origin()`（高德 `place/text` + `citylimit` 硬限定）才可靠，这个 session 之前查机场/其他地标也一直在用这个函数，是港澳场景下地理编码的标准做法。

```
返回 {"recommendations": [... 候选, place / city / category / source ("社区帖子"|"高德POI")/ persona_match_score ...],
"nearby_params": {"origin":{"lng,lat,name}, "hours":int} | None, "clarification_needed": str | None}。
```

agents/route\_agent.py -- 行程/路线 Agent

**职责：**把地点写进 `trip_plan` 的行程节点。模型档位 `MODEL_LIGHT`，纯结构化写入，没接 LLM。两个入口：

| 函数                                                                                                              | 作用                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>run(shared_state, places, time_budget=None, city=None)</code>                                             | 旧接口， <code>places</code> 是地点名字字符串列表，全部按顺序写进同一个占位日期 <code>_PLACEHOLDER_DAY = "day-1"</code> 。 <code>server.py</code> 的 <code>POST /widget-response</code> （ <code>attraction_picker</code> 确认）还在用这个，2026-09-14 重构时保留没动 |
| <code>schedule(shared_state, candidates, city, mode="trip", time_budget_days=1, hours=None, origin=None)</code> | <b>2026-09-14 新增</b> ，统一排时间接口，给编排 Agent 调用                                                                                                                                                                            |

`schedule()` 是这次重构的核心新增：`candidates` 是 `content_agent.run()` 产出的候选（带经纬度的才会被排班，没坐标的直接进 `unscheduled`）。`mode="nearby"`：单日短途，`origin` 是真实起点坐标，贪心选近的凑够 `hours` 预算；`mode="trip"`：按 `time_budget_days` 天数贪心分配，每天默认 09:00 排到 20:00、最多 6 站，**真正新开 day-N 天数**（不再是单一占位日期，`route_agent.run()` 长期悬而未决的“没有真正多日期规划”问题到这里解决了）。到达/结束时间是真推进时间游标算出来的，交通耗时查 `map_tool.route_between()`（2km 阈值切驾车，跟旧版同一个阈值）；天气只查最后一站附近未来 24 小时（避免每站都查一次浪费调用次数）。

内部拆了几个小函数：`_greedy_order()`（贪心选近的，纯几何不算时间）、`_place_day()`（把选好的顺序真实排进一天、算真实到达时间、超时的切进 `unscheduled`）、`_real_leg()`（查两点间真实交通，跟 `run()` 用的 `_estimate_transport()` 是姐妹函数，一个返回格式化字符串，一个返回原始数据方便推进时间游标）。

**已知限制：** `_real_leg()` 内部还是走 `map_tool.route_between()`（连带用到的 `geocode/geo`），对港澳场景两个挨得很近的真实地点偶尔会查询失败（实测澳门大三巴附近几个景点之间出现过“待定（地图查询失败）”，这是 `map_tool.route_between()` 本身的已知限制，不是这次新引入的问题，降级行为（标“待定”、继续排下一站）是既有的、已接受的处理方式。

agents/ota\_hotel\_agent.py -- OTA/酒店 Agent

**职责：**查库存/比价。模型档位 `MODEL_LIGHT`。**酒店这半边 2026-09-13 起接了真实数据**（`hotel_tool.py`，道旅 RollingGo），**机票这半边还是纯占位假数据**——机票比价没有免费的个人开发者可用来源（携程/去哪儿这类都是企业商家合作性质，不对个人开发者开放，调研过没有绕开的办法）。字段补全到“用户选中后能直接落地”的程度：机票候选带 `flight_no / from_ / to / depart_time / arrive_time / status`，酒店候选带 `address / check_in / check_out`（`check_in / check_out` 会用 `date_range` 参数填，格式“开始日期~结束日期”，没传就默认“明天起 2 晚”）。这样 `POST /widget-response`（见 `server.py` 一节）收到用户选中的候选后，能直接拿这些字段调 `trip_plan.add_flight()` / `add_hotel()`，不用现造字段。

`run(shared_state, location, date_range=None, category=None, user_message=None)`：`location` 没传就从 `shared_state["city"]` 兜底（再没有就是 `demo` 默认“澳门”），`user_message` 传了会原样喂给 `hotel_tool.search_hotels()` 当查询意图描述（比拼关键词更准，`orchestrator_agent.py` 的 `booking` 分会把这轮用户原话传进来）；真查失败（key 没配/网络问题）优雅降级成空酒店列表 + `hotel_query_error` 字段，不会抛异常炸穿 `orchestrate()`。

`orchestrate()` 里 `booking` 意图命中时会把这里返回的 `candidates` 分别喂给 `widgets.build_flight_picker_widget()` / `widgets.build_hotel_picker_widget()`（按 `candidates` 里的 `provider_type` 字段分流，见下面 `widgets.py` 一节），机票假数据、酒店真数据混在同一个候选列表里，两个 `widget` 都能出内容。

**已知缺口（2026-09-13 真实端到端测试时发现）：**`orchestrate()` 目前没有从用户消息里提取日期，`date_range` 从来没被传给 `run()`，酒店查询永远用默认的“明天起 2 晚”。真实数据接上之后这个问题第一次变得可见——之前反正是假数据，日期对不对不影响观感；测试时间“9月20-22号”的酒店，LLM 最后收尾组句时诚实地发现拿到的是默认日期的价格，主动跟用户说明了“这不是你问的那两天的价格”（防幻觉提示词生效的证据，没有硬装作数据对上），但功能上确实没做到“按用户说的日期查”。修法待定：可以简单加一段日期正则提取，也可以扩展 `classify_intent` 顺便做实体抽取，需要先决定思路。

agents/exception\_agent.py -- 异常应变 Agent

**职责：提案制**（2026-09-11 合并 `fellow` 的 `codex/integrate-travel-guardian` 分支后定型）——`run()` / `check_weather()` 两条评估路径都只评估、不执行，`requires_confirmation` 恒为 `True`，谁都不直接碰 `trip_plan`；`apply_adjustment()`（2026-09-14 新增）才是“用户确认后真正执行”的那一步。模型档位架构文档里写的是“中等模型”，目前只有 `MODEL_FULL` / `MODEL_LIGHT` 两档，先待定。

| 函数                                                            | 作用                                                                                                                                                                                                                   |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>run(shared_state, event_type, event_detail=None)</code> | 按地点名字匹配 <code>trip_plan</code> 里的行程节点（纯子串匹配， <code>event_detail</code> 是用户整句话），命中就列进 <code>affected_locations</code> ， <b>不删</b> 。 <code>event_verified</code> 恒为 <code>False</code> ——纯粹是“消息里提到了这个地名”，没有验证过是不是真的发生了 |
| <code>check_weather(shared_state, city)</code>                | <b>真查</b> <code>weather_tool.py</code> （和风天气灾害预警），不依赖用户有没有主动提到天气——只要能确定城市就查真实数据。 <code>event_verified</code> 恒为 <code>True</code> ：跟 <code>run()</code> 的关键区别是这条数据来自真实 API，不是子串猜的，但一样不自动改行程                          |
| <code>apply_adjustment(shared_state, place)</code>            | <b>真删</b> ：按地点名字（精确匹配，通常就是前两个函数提案里 <code>affected_locations</code> 的某一项）从 <code>trip_plan</code> 里删掉对应节点，可能跨天/命中多个。返回 <code>{"cancelled": [...], "count": N}</code> ， <code>count=0</code> 不算错误，可能是提案之后行程又被改过        |

`run()` / `check_weather()` 返回同一套评估契约：  
`has_warning / needs_replan / requires_confirmation / event_verified / suggested_adjustment`（`run()` 版本另外还有 `affected_locations`）。`apply_adjustment()` 是单独一套“执行结果”契约，不跟评估契约混在一起。

`apply_adjustment()` 底层调的是 `trip_plan.py` 新增的两个函数: `cancel_stop_by_id(trip_plan, node_id)` (按 `node_id` 精确删一个) 和 `cancel_stops_by_place(trip_plan, place)` (按地点名字删, 可能命中多个) ——这两个是对 `trip_plan.remove_stop(day_plan, node_id)` 的封装, 调用方不用自己先找是哪一天, 直接传整个 `trip_plan` 和 `node_id/地点名字` 就行。

**这个函数目前没有被 `orchestrate()` 调用**——“怎么判断用户确认了、什么时候该调 `apply_adjustment()`”这个触发逻辑还没接 (见文末“待接事项”), 2026-09-14 这次只是先把“执行”这个接口本身准备好, 方便编排 Agent 以后调用。

`check_weather()` 的行为: - 有真实预警, `needs_replan` 会不会标 `True` 取决于预警等级 (`severity`) 够不够到 `severe / extreme` (常量 `_NEEDS_REPLAN_SEVERITY`) ——**但不管等级多高都不会自动改 `trip_plan`**, 这只是给编排 Agent 一个“要不要用更紧迫的语气跟用户说”的信号, 不是执行开关 (2026-09-11 之前的版本在 `severe / extreme` 时会自动清空当天行程, 合并 `fellow` 分支时改掉了, 统一成提案制) - 没有预警返回 `has_warning: False`, 是正常情况; `weather_tool` 查询本身失败 (key 没配/地名查不到/网络问题) 不会抛异常炸穿调用方, 优雅降级返回带 `error` 字段的结果——跟 `route_agent.py` 处理 `map_tool` 查询失败是同一个思路 - `orchestrator_agent.py` 里 `exception` 意图命中时会额外跑一次: 从用户消息里提取城市 (`_KNOWN_CITIES = (“澳门”, “香港”)`), 跟 `content_agent.extract_city` 同一套子串匹配思路, 独立一份没有互相 `import`, 两个模块本来就不该耦合), 提取到了才调用 `check_weather()`, 结果挂在 `results[“exception”][“weather_check”]` 里, 最后收尾的 LLM (或 `restaurant` 命中时的字符串拼接) 会把这个提案说给用户听 - **不写 `trip_plan.weather_alerts`**: 提案制意味着“记录下来”本身也算一种执行, 所以这版故意不调 `trip_plan.add_weather_alert()` ——`GET /trip` 暂时看不到这些真实预警, 等“确认后执行”那个接口做出来了再一并把写入这一步补上

widgets.py -- 展示插件 (右侧聊天框富交互组件)

**职责:** 右侧聊天框里除了纯文字回复之外的富交互小组件 (帖子列表、景点勾选等), 跟 Agent/编排逻辑解耦——组件只负责“从真实候选集里挑/怎么展示”, 不重新跑检索或编内容。

**设计原则:** - 候选集永远来自真实代码算出来的数据 (目前是 `content_agent.run()` 的 `recommendations`, 已经是 `store.query_similar_posts()` 按人格相似度排过序的真实结果), 不允许凭空捏造。 - 要用 LLM 做“选择/排序”时, LLM 只能从候选集里选 ID, 不允许自己编内容; `_llm_select()` 会校验 LLM 返回的 ID 是否都在候选集里, 筛掉任何编造的 ID (防幻觉), LLM 调用/解析失败时优雅降级成“按候选集原有顺序截取前 N 个”, 不影响整体流程。 - 每个 widget 统一格式 `{“widget”: “<type>”, “data”: {...}}`, 前端按 `widget` 类型分发渲染, 渲染逻辑完全不用管数据是怎么算出来的。 - 模型档位: `_llm_select()` 内部用 `llm_tool.MODEL_LIGHT` (选择/排序是结构化任务, 不需要强模型)。

| 函数                                                                                                      | 作用                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>build_post_list_widget(candidates, top_k=3, use_llm_rerank=False)</code>                          | 帖子展示插件: 默认直接截取候选集前 <code>top_k</code> 个 (候选集已经是真实排序过的, 不用额外调 LLM); 传 <code>use_llm_rerank=True</code> 走 LLM 二次筛选                                                                                                                                                                                            |
| <code>build_attraction_picker_widget(candidates, max_select=3, pool_size=6, use_llm_rerank=True)</code> | 景点选择插件: 从候选里选出 <code>pool_size</code> 个放进“可选池”返回给前端, 用户从池子里最多勾 <code>max_select</code> 个; 默认走 LLM 精选候选池 (LLM 只能选真实 ID)                                                                                                                                                                                      |
| <code>build_flight_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>     | 机票选择插件: 跟 <code>build_attraction_picker_widget</code> 同一套“可选池 + 前端勾选 + 结果回传”契约, 只是机票通常只订一个, 默认 <code>max_select=1</code> ; 从 <code>candidates</code> 里只挑 <code>provider_type == “flight”</code> 的, 走 LLM 精选出 <code>pool_size</code> 个放进候选池; 筛出来是空的就返回 <code>None</code> (调用方按 <code>None</code> 跳过, 不展示空插件) |
| <code>build_hotel_picker_widget(candidates, max_select=1, pool_size=3, use_llm_rerank=True)</code>      | 酒店选择插件: 跟 <code>build_flight_picker_widget</code> 同一个模式, 只挑 <code>provider_type == “hotel”</code> 的; 同样可能返回 <code>None</code>                                                                                                                                                                               |

**依赖:** - `build_post_list_widget / build_attraction_picker_widget` 用 `place` 字段 (2026-09-14 起从 `post_id` 换成 `place`) 做 `_llm_select` 的 ID 校验——`content_agent.run()` 的候选自那次重构起有两种来源 (社区帖子带 `post_id`, 达人 Agent 新扩展出的高德 POI 不带), `place` 是两种来源都有的字段, 继续用 `post_id` 会导致 POI 来源的候选在 LLM 精选这一步被静默漏掉。 - `build_flight_picker_widget / build_hotel_picker_widget` 的候选 (`ota_hotel_agent.run()` 的 `candidates`) 没有独立 `id` 字段, 改用 `name` 字段当 `_llm_select` 的校验 key——跟 `place / post_id` 是同样的作用, 只是换了个字段名。

**四个 widget 分两种交互模式：** `post_list` 是纯展示（截取直接列出来）； `attraction_picker` / `flight_picker` / `hotel_picker` 是"可选池 + 前端勾选 + 结果回传"—— `options` + `max_select` 是统一契约，机票/酒店默认 `max_select=1`（通常只订一个），景点默认 `max_select=3`（可以多选几个）。

**现状：**四个 widget 都已经真正接进 `web/index.html` 渲染，`POST /widget-response` 接口也做完了（见 `server.py` 一节）——右侧聊天框里能看到真实的卡片/勾选交互，勾选点"确认选择"会真的把机票/酒店写进 `trip_plan`、把景点排进路线，不再是"传了但没人用"的状态。

**待接事项：** - 讨论过的其他 widget 想法还没做：反馈评分插件、异常变更确认插件、人格问卷引导插件

schedule\_widgets.py -- 左侧日程面板展示组件

**职责：**服务对象是左侧实时日程面板（`server.py` 的 `GET /trip`，`web/index.html` 目前的文字列表），跟右侧聊天框的 `widgets.py` 是同一个"数据/展示分离"思路——这里只算"给前端画图/排版用的数据"，不产出任何 HTML/画布逻辑，真正怎么画留给前端。模型档位：不需要 LLM，纯确定性计算（按天/类型分配颜色 + 查真实地理数据）。三个 widget：

| 函数                                                           | 作用                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>build_trip_map_widget(trip_plan_obj, city=None)</code> | 地图组件：把每天的景点/餐饮节点标进地图（同一天同一个颜色，颜色按"第几天"从固定调色板分配，超过 8 天循环复用），并把当天节点依次连成一条 <b>真实路线</b> （查真实路况，不是直线连线，颜色跟当天 <code>markers</code> 一致）；机票起降机场、酒店地址也各标一个点，用固定颜色（不参与按天上色，因为它们不属于"某一天"）。多城市行程会按地理距离自动 <b>分帧</b> （见下），不再是所有点挤在同一张地图里                                   |
| <code>build_day_timeline_widget(trip_plan_obj)</code>        | 每日行程时间线：把每天的节点按时间顺序整理成竖排列表，按天分模块，模块颜色跟 <code>build_trip_map_widget</code> 里同一天的颜色对齐，方便地图和列表对照着看                                                                                                                                                              |
| <code>build_booking_panel_widget(trip_plan_obj)</code>       | 机票/酒店面板：原样呈现 <code>trip_plan</code> 里已经落地确认的 <code>flights</code> / <code>hotels</code> 记录——注意这跟 <code>widgets.py</code> 的 <code>build_flight_picker_widget</code> / <code>build_hotel_picker_widget</code> 不是一回事，那两个是"聊天框里给用户挑的候选推荐"，这里是"已经确认、要显示在左侧行程里的记录" |

`build_trip_map_widget` 返回结构（**分帧版**，2026-09-11 起替换了原来的扁平 `markers` / `routes` 结构）：

```
{
  "widget": "trip_map",
  "data": {
    "frames": [
      {
        "frame_id": "frame-1",
        "center": { "lng": ..., "lat": ... },
        "markers": [
          {
            "day": ...,
            "color": ...,
            "place": ...,
            "node_id": ...,
            "type": "attraction|meal|airport|hotel",
            "lng": ...,
            "lat": ...
          },
          ...
        ],
        "routes": [
          {
            "day": ...,
            "color": ...,
            "coordinates": [
              [ "lng", "lat" ],
              ...
            ]
          },
          ...
        ],
        "unclustered_airports": [
          {
            "day": ...,
            "color": ...,
            "place": ...,
            "node_id": ...,
            "type": "airport",
            "role": ...,
            "flight_no": ...,
            "status": ...,
            "lng": ...,
            "lat": ...
          },
          ...
        ],
        "days_legend": [
          { "date": ..., "color": ... },
          ...
        ],
        "geocode_failures": [
          { "place": ..., "reason": ... },
          ...
        ],
        "route_failures": [
          { "day": ..., "from": ..., "to": ..., "reason": ... },
          ...
        ]
      },
      ...
    ],
    "unclustered_airports": [
      {
        "day": ...,
        "color": ...,
        "place": ...,
        "node_id": ...,
        "type": "airport",
        "role": ...,
        "flight_no": ...,
        "status": ...,
        "lng": ...,
        "lat": ...
      },
      ...
    ],
    "days_legend": [
      { "date": ..., "color": ... },
      ...
    ],
    "geocode_failures": [
      { "place": ..., "reason": ... },
      ...
    ],
    "route_failures": [
      { "day": ..., "from": ..., "to": ..., "reason": ... },
      ...
    ]
  }
}
```

- **为什么要分帧：**早期版本是所有 marker 挤进同一张地图，跨城市行程（比如"杭州玩三天+机票飞北京"）一旦机票起降机场跟行程主城市离得远，地图会被迫缩到能同时装下两座城市的比例尺，近处的景点全部挤成一个点，什么都看不清。改成按地理距离把 marker 分组，每组（帧）各自算自己的地图范围/比例尺，前端各画一个独立的 `AMap.Map` 实例，近处的点就能放大到看得清的程度
- **分帧算法**（`schedule_widgets.py` 里的 `_cluster_by_distance` + `_haversine_m`）：对当天的景点/餐饮/酒店 marker 用 `haversine` 距离做贪心单链聚类，阈值 `_FRAME_CLUSTER_THRESHOLD_M = 50_000`（50km）——两个点只要有一条  $\leq 50\text{km}$  的链路就会被分进同一帧，超过阈值的自动分到不同帧
- **机票起降机场不参与聚类本身**，而是聚类结束、每帧的中心点算出来之后，拿机场坐标跟每一帧的中心点比距离，落在 `_AIRPORT_ATTACH_THRESHOLD_M = 80_000`（80km）以内就并入最近的那一帧（进 `frames[i].markers`），否则单独放进

- `unclustered_airports`（不画在任何一帧里，避免为了容纳一个远处的机场把某一帧的比例尺又拉爆）——这是用户明确要求的规则：“起点和终点不算，直接忽略就行”
- 景点/餐饮 `marker` 的 `day / color` 按天分配；机票（`type: "airport"`，额外带 `role: "depart"|"arrive"`、`flight_no`、`status`）和酒店（`type: "hotel"`，额外带 `name / check_in / check_out`）的 `day` 固定是 `None`，颜色分别固定为深灰蓝/棕色（`_AIRPORT_COLOR / _HOTEL_COLOR`），不占用按天调色板
  - 每帧内 `routes` 的坐标是把 `map_tool.route_between()` 返回的多段 `polyline` 展平成一条连续的 `[lng, lat]` 序列，直接给前端画线；一条路线按起点锚点归到最近的帧里；查询失败的那一段跳过并记进 `route_failures`，不影响其他天/其他段
  - **字段约定**： `flights` 的 `from_ / to`、 `hotels` 的 `address` 建议存能被地理编码识别的地名/地址（比如“杭州萧山国际机场”），不建议只存三字码（比如 `"HGH"`）——高德地理编码认不出机场三字码，会直接进 `geocode_failures`

**现状**：三个 widget 已经接进 `server.py` 的 `GET /trip`，`web/index.html` 用高德 JS API 真的把地图画出来了（见 `server.py` + `web/index.html` 一节）；已经用真实 `AMAP_KEY` 端到端验证过，标点/连线/机票酒店坐标都是真实经纬度。

**现状/限制**： - 跟 `route_agent.py` 一样依赖 `AMAP_KEY`（后端查经纬度/路线用的那个，注意不是前端画图用的 `AMAP_JS_KEY`），没配置时 `markers / routes` 会是空的、`*_failures` 里全是查询失败记录（不会导致整体报错，自测+真实端到端联调都验证过这个降级路径） - `route_agent.py` 里已经算过一次相邻站点的交通方式/耗时，这里为了拿到完整 `polyline` 又独立查了一次路线（避免“展示层”反过来依赖某个 Agent 的内部计算结果），会有一点重复的高德 API 调用，demo/比赛规模的免费额度足够用，不是问题 - `POST /widget-response` 确认机票/酒店后会真的调 `trip_plan.add_flight() / add_hotel()`，`build_booking_panel_widget` / 机票酒店 `marker` 现在跑起来是有真实数据的（端到端联调过：选中机票+酒店 → `/trip` 能看到确认记录） - **机票起降机场地理编码不传 `city`**： `build_trip_map_widget(trip_plan_obj, city=...)` 的 `city` 参数只用于景点/餐饮/酒店这些“行程主城市内”的地点，机场坐标查询故意不传 `city` ——接真实 key 测试时踩到过一坑：查“上海虹桥国际机场”时如果传了 `city="杭州"`，高德会把它强行匹配成杭州萧山机场附近的结果（两个机场坐标几乎重合），因为出发/到达机场天然可能分属两个不同城市，用行程主城市限定反而会会查错

main.py -- 入口脚本

**职责**：纯胶水代码，不包含任何 Agent 逻辑。`from agents.orchestrator_agent import new_shared_state, orchestrate` 之后直接 `re-export`，`server.py` 还是 `import main` 调这两个函数，完全不用改。`__main__` 里是完整的端到端 demo（推荐 → 排路线 → 异常应变 → 打印最终行程）。

store.py -- 达人社区内容 + 历史记录

**职责**：达人社区的图文帖子存储 + 检索（两阶段检索第一阶段：按人格向量相似度筛选），以及行程反馈历史记录。

**存储技术**：Chroma 向量库（`orchestrator/data/chroma/`，本地文件持久化）存帖子；JSON Lines 文件（`orchestrator/data/history.jsonl`）存历史记录；图片本地落盘（`orchestrator/data/images/<post_id>/`），帖子里只存相对路径。

| 函数                                                                   | 作用                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>save_image(post_id, filename, image_bytes)</code>              | 写一张图，返回存进 <code>content["images"]</code> 的相对路径                                                                                                                                                                                                                       |
| <code>resolve_image_path(相对路径)</code>                                | 相对路径转本地绝对路径，读图用                                                                                                                                                                                                                                                      |
| <code>add_post(post_id, persona_vector, content)</code>              | 存一条帖子。 <code>content</code> 字段见下表                                                                                                                                                                                                                                    |
| <code>query_similar_posts(persona_vector, top_k=5, city=None)</code> | 两阶段检索第一阶段：按人格向量相似度找候选帖子，返回时带 <code>similarity_score</code> ； <code>verified_local=True</code> 的帖子会被优先加权（见下面说明）；传了 <code>city</code> 会在 Chroma 查询阶段就用 <code>where={"city": city}</code> 精确过滤，不是“先按相似度截 <code>top_k</code> 再筛掉不 <code>match</code> 的”（那样可能筛到结果不够甚至是空的） |
| <code>find_posts_by_place(place)</code>                              | <b>2026-09-14 新增</b> ：按地点名字精确匹配查帖子（Chroma metadata 过滤，不是向量相似度），给达人 Agent 的“口碑复核”用（ <code>content_agent._nearby_plan()</code> ）。找不到返回空列表，是正常情况——真实高德 POI 名字跟帖子里手的地点名字经常对不上，命中率本                                                                                       |

| 函数                                                 | 作用                                                                                                                                                             |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                    | 来就不高。每条额外带 <code>persona_vector</code> 字段（要显式 <code>include=["embeddings"]</code> 才拿得到，默认 <code>Chroma .get()</code> 不返回 <code>embedding</code> ），给口碑复核算余弦相似度用 |
| <code>delete_post(post_id)</code>                  | 删帖                                                                                                                                                             |
| <code>log_history(user_id, trip_id, record)</code> | 追加一条行程反馈记录                                                                                                                                                     |
| <code>get_history(user_id=None)</code>             | 读历史记录，不传 <code>user_id</code> 读全部                                                                                                                              |

`content` 字段（`add_post` 的第三个参数）：

| 字段                                                                              | 说明                                                                                                    |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>place / time_slot / avg_cost / rating / avoid_tips / verified_trip</code> | 结构化字段，对齐 <code>agent-interfaces.md</code> 达人 Agent 输出契约                                               |
| <code>caption</code>                                                            | 帖子正文文字，可选                                                                                             |
| <code>images</code>                                                             | 图片相对路径列表，可选，由 <code>save_image()</code> 生成                                                            |
| <code>tags</code>                                                               | 标签列表，可选，跟 <code>images</code> 一样序列化存（ <code>tags_json</code> ），查询时还原回列表                               |
| <code>city / category / post_type / address / verified_local</code>             | 港澳达人数据库用的字段（见 <code>import_hk_macau_data.py</code> 一节），<br><code>verified_local=True</code> 表示本地人认证来源 |

（`query_similar_posts()` / `find_posts_by_place()` 内部共用一个 `_restore_metadata()` 私有函数做 `images_json / tags_json` 反序列化，2026-09-14 从 `query_similar_posts()` 里抽出来的，别在新函数里再写一遍。）

**"本地人认证"优先加权是怎么做的：**`query_similar_posts()` 先按 `top_k * 3` 多捞一些候选（不是只捞 `top_k` 个），把 `verified_local=True` 的帖子相似度乘一个固定系数 `_VERIFIED_LOCAL_BOOST = 1.15`，再按加权后的分数重新排序截到 `top_k`。这个"先多捞再重排"是必须的——如果直接对 `Chroma` 已经按原始距离截好的 `top_k` 结果加权，加权只能在这几条里面换个顺序，换不进被漏掉的本地人帖子，加权就没意义了。

**待定/限制：**- 内容相关性排序（两阶段检索第二阶段）没实现，留给调用方在 `query_similar_posts()` 的候选集里自己排 - 图片存本地文件系统，部署到 `ModelScope Studio` 时要换成对象存储（OSS/CDN），`images` 字段到時候存 URL 而不是本地路径，上层调用方式不变 - `_VERIFIED_LOCAL_BOOST` 系数（1.15）是拍的，没有做过效果对比调优，觉得加权效果不明显/太强可以直接改这个常量

standardize\_hk\_macau\_data.py + import\_hk\_macau\_data.py -- 港澳达人数据库

**背景：**组员手动整理了一份港澳（澳门/香港）达人推荐/避坑数据，存在仓库根目录的 `数据库原始.xlsx` 里，来源分两种：- **本地人认证**（Excel 里的"本地人澳门"/"本地人香港" sheet）：饮食/景点/Tips 分类推荐 + 单独一张避坑表 - **小红书**（"小红书澳门"/"小红书香港" sheet）：褒贬都有的普通帖子，不算本地人认证来源

这两种来源对应 `store.py` 里新加的 `verified_local` 字段——本地人认证的帖子在检索时会被优先加权（见 `store.py` 一节），小红书来源不加权。这个字段是原始数据自带的真实来源标记（组员整理的时候已经分好表了），不是靠算法从内容"判断"出来的。

**两步流程**（分开是为了标准化出错时不用重新灌一次向量库）：

| 脚本                                        | 作用                                                                                                                               |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>standardize_hk_macau_data.py</code> | 读 <code>数据库原始.xlsx</code> ，标准化字段 + 编号，写出 <code>data_sources/hk_macau_posts.json</code> 。不碰 <code>Chroma</code>                   |
| <code>import_hk_macau_data.py</code>      | 读标准化 JSON，用 <code>persona.infer_post_persona_vector()</code> 给每条帖子算一个人格向量，调 <code>store.add_post()</code> 灌进 <code>Chroma</code> |

**编号规则**（城市前缀 + 类型字母 + 三位数字，类型内连续编号，不区分本地人/小红书来源）：

|      | 澳门•推荐               | 澳门•避坑           | 香港•推荐               | 香港•避坑        |
|------|---------------------|-----------------|---------------------|--------------|
| 编号格式 | MF/MS/MT + 三位数字     | MN + 三位数字       | HF/HS/HT + 三位数字     | HN + 三位数字    |
| 示例   | MF001, MS001, MT001 | MN001, MN002    | HF001, HS001, HT001 | HN001, HN002 |
| 说明   | 饮食/景点/Tips, 正常分类    | 全部统一放这里，不分饮食/景点 | 同左                  | 同左           |

解读：M=Macau, H=Hong Kong（城市前缀）；F=Food, S=Sight, T=Tips（推荐分类）；N=No（避坑专用，统一放一起不细分）。

**原始表格解析上的坑：**Excel 里不同分类小节的列不是对齐的（比如“本地人香港”的 Tips 小节没有单独的“名称”列，内容直接写在名称那一格；“本地人香港”饮食小节比“本地人澳门”多一列“推荐人”），standardize\_hk\_macau\_data.py 按每个小节自己的表头动态取列，不能假设固定列位置——这个坑是解析的时候踩出来的，写死列位置会导致数据错位。

**当前数据量**（229 条，跑 python standardize\_hk\_macau\_data.py 会打印这个分布）：

| 前缀 | 条数              | 前缀 | 条数             |
|----|-----------------|----|----------------|
| MF | 50（本地人30+小红书20） | HF | 19（本地人9+小红书10） |
| MS | 50              | HS | 20             |
| MT | 50              | HT | 20             |
| MN | 10              | HN | 10             |

**已知数据缺口：**香港饮食类本地人认证只有 9 条，不是目标的 10 条——原始表格里就是这样，组员应该是漏填了一条，不是解析脚本的 bug，不影响其他数据使用。

**怎么重新跑：**

```
python orchestrator/standardize_hk_macau_data.py # 改了 Excel 或标准化逻辑之后跑
python orchestrator/import_hk_macau_data.py      # 改了人格反推逻辑，或者清空过 orchestrator/data/chroma 之后跑
```

import\_hk\_macau\_data.py 用的是 upsert（store.add\_post() 内部调的），同一个 id 重复导入会覆盖不会重复，可以放心重跑。

**待接事项：**- 避坑类帖子（MN / HN）反推出来的人格向量是中性默认值（见 persona.py 一节的已知局限），检索排名上不占优势也不吃亏，更合理的做法（不管人格匹配度、只要地点/类型对上就该出现）属于两阶段检索第二阶段，还没实现 - verified\_local 优先加权的系数（1.15）没有做过 A/B 效果对比，是拍的经验值 - content\_agent.py 的两阶段检索第二阶段接上之后，category（饮食/景点/Tips）字段可以用来做更精细的“用户问吃的就优先出饮食类”这种过滤，现在还没用上

trip\_plan.py -- 旅游计划（行程/机票/酒店/天气异常）

**职责：**一次完整旅行的所有结构化信息，由多个 Agent 共同维护，最终汇总用于呈现。

**顶层结构：**

```
trip_plan = {
    "trip_id": "...",
    "flights": [...],          # OTA/酒店 Agent 维护
    "hotels": [...],           # OTA/酒店 Agent 维护
    "weather_alerts": [...],   # 异常应变 Agent 维护
    "days": {日期: day_plan}, # 行程 Agent 生成, 异常应变 Agent 增删改
}
```

**景点通勤+餐饮的核心设计：**每天一个“链表”，但不是指针式链表，而是**节点字典 + id 引用**（`day_plan["nodes"]` 是 `{node_id: node}`，每个节点有 `next_id` 指向下一个节点的 id，`day_plan["head_id"]` 是链表头）。这样设计是因为：- 异常应变 Agent 改行程时只需要动被影响的那一个节点、接一下前后 `next_id`，不用重发整天的数组 - 本质是个普通 dict，直接能存 JSON/传 HTTP，不需要额外的链表↔数组转换逻辑

节点字段：`type`（`"attraction" | "meal"`）、`place`、`arrival_transport`、`arrival_time`、`end_time`、`next_id`；`meal` 类型节点多一个 `meal_type`（早餐/午餐/晚餐/夜宵）。

| 函数                                                                                                                        | 作用                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>new_trip_plan(trip_id)</code>                                                                                       | 造一个空旅游计划                                                                                                                                                                                                 |
| <code>add_flight / add_hotel / add_weather_alert</code>                                                                   | 往对应列表追加一条记录                                                                                                                                                                                              |
| <code>get_or_create_day(trip_plan, date)</code>                                                                           | 拿到某天的 <code>day_plan</code> ，不存在就新建                                                                                                                                                                      |
| <code>add_stop(day_plan, node_id, type_, place, arrival_transport, arrival_time, end_time, after_id=None, **extra)</code> | 插入一个节点， <code>after_id=None</code> 插到最前面，否则插到该节点后面                                                                                                                                                       |
| <code>remove_stop(day_plan, node_id)</code>                                                                               | 删除节点，自动重连前后节点。低层原语，调用方要先自己找到 <code>day_plan</code>                                                                                                                                                       |
| <code>cancel_stop_by_id(trip_plan, node_id)</code>                                                                        | 2026-09-14 新增：按 <code>node_id</code> 删节点，不用先自己找是哪一天——遍历 <code>trip_plan["days"]</code> 定位到再删，返回被删节点内容（带 <code>date / node_id</code> ）或 <code>None</code>                                                 |
| <code>cancel_stops_by_place(trip_plan, place)</code>                                                                      | 2026-09-14 新增：按地点名字（精确匹配）删节点，可能跨天/命中多个，返回被删节点列表。这两个是 <code>remove_stop()</code> 的高层封装，给 <code>exception_agent.apply_adjustment()</code> 用，也是编排 Agent"确认后执行"时应该调的接口，不建议跳过封装直接调 <code>remove_stop()</code> |
| <code>patch_stop(day_plan, node_id, **fields)</code>                                                                      | 只改某个节点的部分字段，不动链表结构                                                                                                                                                                                       |
| <code>day_stops(day_plan)</code>                                                                                          | 把链表还原成有序数组，渲染/地图面板用                                                                                                                                                                                      |
| <code>render(trip_plan)</code>                                                                                            | 汇总成最终展示结构（ <code>flights / hotels / weather_alerts / days</code> ，每天的 <code>stops</code> 已经是有序数组）                                                                                                        |

**现状：**已经接进 `main.py` 的 `shared_state["trip_plan"]`，`route_agent` 会往里面加节点。`exception_agent` 现在是提案制，不会自己删节点——真要删得靠编排 Agent 调 `exception_agent.apply_adjustment()`（内部调 `cancel_stops_by_place()`），但这一步还没接进 `orchestrate()`（见“待接事项”）。目前所有节点都写进同一个占位日期 `"day-1"`（`main.py` 里的 `_PLACEHOLDER_DAY`），还没做真正的多日期规划。

persona.py -- 人格变量

**职责：**定义“旅游人格”的具体维度结构，供达人/行程/OTA Agent 读取，反馈闭环用来校准。

**维度设计**（量化的维度都用了 0~1 连续分数，不是纯标签）：

| 维度                         | 归属                                    | 类型             | 说明                                                 |
|----------------------------|---------------------------------------|----------------|----------------------------------------------------|
| <code>taste</code>         | <code>stable_traits</code> （跨场景稳定）    | 固定词表 multi-hot | 菜系标签，词表见 <code>_TASTE_VOCAB</code>                 |
| <code>novelty_score</code> | <code>stable_traits</code>            | float 0~1      | 0=稳妥大众，1=小众冒险                                      |
| <code>pace_score</code>    | <code>scenario_traits</code> （按场景分开存） | float 0~1      | 0=完全休闲，1=完全特种兵                                     |
| <code>budget_score</code>  | <code>scenario_traits</code>          | float 0~1      | 0=穷游，1=公务舱/总统套房/租车这种享受型消费； <b>硬过滤字段，不进相似度向量</b> ，用 |

| 维度             | 归属              | 类型                                         | 说明                         |
|----------------|-----------------|--------------------------------------------|----------------------------|
|                |                 |                                            | budget_filter_ok() 按数值区间过滤 |
| social_mode    | scenario_traits | 分类<br>( solo / couple / family / friends ) | 并列模式非程度光谱，向量里按 one-hot 编码  |
| interest_theme | scenario_traits | 固定词表 multi-hot                             | 词表见 _INTEREST_VOCAB        |

scenario\_traits 按"本次旅行模式" ( business\_trip / vacation / family / solo\_adventure ...) 分开存，同一个人不同场景下这些值可以完全不同。

| 函数                                                                                           | 作用                                                                                                                                       |
|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| new_persona(user_id)                                                                         | 造一份空人格                                                                                                                                   |
| bootstrap_from_onboarding(user_id, scenario, answers)                                        | 冷启动：从一份问卷答案组出初始 persona                                                                                                                  |
| set_scenario_traits(persona, scenario, **fields)<br>/ get_scenario_traits(persona, scenario) | 读写某场景的特质，写入时会校验词表 + clamp 数值到 0~1                                                                                                        |
| update_stable_traits(persona, **fields)                                                      | 读写跨场景稳定特质，同样有校验                                                                                                                          |
| apply_feedback(persona, scenario, dimension, new_value, reason="")                           | 按维度校准，写入 + 留痕 ( feedback_log )。"该改成什么值"由调用方决定，这里只负责写入                                                                                    |
| budget_filter_ok(a_score, b_score, max_diff=0.25)                                            | 判断两个 budget_score 是否够接近，硬过滤专用，独立于相似度向量                                                                                                   |
| compute_persona_vector(persona, scenario)                                                    | 拼出向量：[novelty_score, taste multi-hot, pace_score, social_mode one-hot, interest_theme multi-hot]，固定 21 维 (词表大小变了维度也会变，加了"葡国菜"后从 20 变 21) |
| infer_post_persona_vector(scenario, tags=None, avg_cost=None, category=None)                 | 港澳达人数据库导入专用：这些帖子是组员手动整理的真实内容，没有"发帖人自己的问卷答案"，用标签/人均/分类反推一个大致合理的人格向量，见下面单独说明                                                               |
| cosine_similarity(vector_a, vector_b)                                                        | <b>2026-09-14 新增：</b> 两个人格向量的余弦相似度 (0~1，向量分量都非负)，给达人 Agent 的"口碑复核"用——拿当前用户的人格向量跟"评价过某地点的帖子"的人格向量比对。维度对不上直接返回 0.0，不抛异常                    |

**infer\_post\_persona\_vector() 是怎么反推的** (启发式估计，不是精确画像)： - \_TAG\_TO\_TASTE / \_TAG\_TO\_INTEREST 两个关键词映射表，对标签文本做子串匹配，匹配上就计入对应的 taste / interest\_theme；匹配不上的标签 (比如"三代传承""400年历史"这种描述性但不构成人格维度信号的) 直接丢弃，不强行凑 - category (饮食/景点) 也会补一条弱信号：饮食→"美食探店"，景点→"人文历史" - avg\_cost 线性换算成 budget\_score (0~250 元映射到 0~1，250 元封顶)，换算公式很粗糙，不是精确定价模型 - 标签里出现"小众/冷门/隐世"这类词 → novelty\_score 偏高 (0.75)；出现"网红/热门/必打卡" → 偏低 (0.3)；都没有就是中性 0.5 - social\_mode 故意不猜——单条帖子看不出"适合独行/家庭/朋友"，留空对应 compute\_persona\_vector() 里那段 one-hot 全 0，这是合法的"未指定"编码 - **已知局限：**避坑 ( 避雷理由 ) 类帖子在标准化数据里没有 tags / avg\_cost / category，反推出来的人格向量是中性默认值 ( novelty=0.5, budget=0.5，无 taste/interest)，跟任何用户人格的相似度都不会特别高也不会特别低——这类帖子更适合"不管人格匹配度、只要地点/类型对上就该出现"，属于两阶段检索第二阶段 (内容相关性排序) 该管的事，那部分还没实现

**待定/限制：** - compute\_persona\_vector() 是数值特征拼接，不是真实语义 embedding，以后接真实 embedding 模型可以整体替换掉函数体，调用方 ( store.py ) 接口不用变 - 改 \_TASTE\_VOCAB / \_INTEREST\_VOCAB 词表会改变向量维度，如果本地 Chroma 已经有存量数据，改词表前要清掉 orchestrator/data/chroma 重新灌数据，不然新旧向量维度对不上 (这个坑已经踩过一次，这次加"葡国菜"也重新灌了一次数据) - 从一条用户反馈文本判断"该把哪个维度调成什么值"的算法没实现，留给编排/达人 Agent 决定

server.py + web/index.html -- 呈现层 (网页)

**职责：**把 `main.py` 的编排 Agent、`widgets.py`、`schedule_widgets.py` 包成 HTTP 接口 + 真实渲染，配一个两栏网页——左边实时日程（地图+每日时间线+机票酒店面板），右边聊天框（文字回复+四个插件卡片）。这是目前唯一“从对话到落地预订”整条链路能真的跑通的地方。

**运行方式：**

```
pip install flask python-dotenv
python orchestrator/server.py
```

然后浏览器打开 `http://127.0.0.1:5000`。地图能不能真的画出来，取决于 `.env` 里三个高德 key 有没有配全（见“环境准备”一节）；没配的话页面照样能跑，只是地图是空的、控制台会有一条警告。

**会话隔离**（2026-09-11 合并 fellow 分支后新增）：每个浏览器发一个 `travel_session` cookie（32 位十六进制随机值），后端靠这个 cookie 从 `session_store.py`（SQLite）读写各自独立的 `shared_state`，不再是全局共享一份——不同人/不同浏览器打开不会互相覆盖行程和对话记录，但仍然是单进程本地 demo，没有账号登录体系。

**接口**（都是同一个 Flask app 提供，同源不用处理 CORS）：

| 接口                                                        | 作用                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /                                                     | 返回 <code>web/index.html</code> ，服务端会把文件里的 <code>__AMAP_JS_KEY__</code> / <code>__AMAP_JS_SECURITY_CODE__</code> 占位符换成 <code>.env</code> 里的真实值再返回（这两个 key 不写进 git 里的 html 文件，跟其他密钥一样只活在 <code>.env</code> ）                                                                                                                                                                                                                                                               |
| GET /session                                              | 页面刷新/重新打开时用来恢复对话历史和待处理 widgets（ <code>pending_widgets</code> ），前端 <code>restoreSession()</code> 调这个                                                                                                                                                                                                                                                                                                                                                                      |
| GET /trip                                                 | 返回 <code>trip_plan.render()</code> 的行程 + 额外几个字段 <code>trip_map</code> / <code>day_timeline</code> / <code>booking_panel</code> （ <code>schedule_widgets.py</code> 那三个函数的输出）                                                                                                                                                                                                                                                                                              |
| POST /chat                                                | body 传 <code>{"message": "...}"</code> ，内部调 <code>main.orchestrate()</code> ，返回编排 Agent 的输出（ <code>chat_reply</code> / <code>community_panel</code> / <code>map_panel</code> / <code>widgets</code> / <code>restaurant_evidence</code> ）。 <b>港澳附近游现在也走这个接口</b> （见 <code>agents/orchestrator_agent.py</code> 一节），不用单独的 <code>/nearby-plan</code>                                                                                                                          |
| POST /widget-response                                     | body 传 <code>{"widget": "attraction_picker" "flight_picker" "hotel_picker", "selected": [...]}</code> ， <code>selected</code> 是前端从对应 widget 的 <code>options</code> 里原样拿到的候选对象。服务端会先校验这个 widget 是不是这个 session 当前真的“待处理”（ <code>pending_widgets</code> ），选中项是不是真的在候选池里、有没有超过 <code>max_select</code> 、有没有重复。景点选中后调 <code>route_agent.run()</code> （旧接口，见 <code>route_agent.py</code> 一节）排进行程；机票/酒店选中后调 <code>trip_plan.add_flight()</code> / <code>add_hotel()</code> 确认预订 |
| POST /nearby-plan / POST /nearby-weather / GET /nearby.js | <b>2026-09-14 起下线</b> （返回 501）：港澳附近游并入了达人 Agent，走 <code>POST /chat</code> 就行。这三个端点+ <code>nearby_planner.py</code> 一起被吸收删除了，端点本身保留但明确返回“已下线”，避免旧前端代码发请求后卡死等结果； <code>web/nearby.js</code> / <code>web/index.html</code> 里那套独立表单+专属地图渲染这版还没跟着改，是已知的前端配套缺口，下一步要做                                                                                                                                                                                                           |

**前端逻辑**（`web/index.html`，原生 HTML/CSS/JS + 高德地图 JS SDK，没引入前端框架）：

- 发消息 → 调 `/chat` → 显示文字回复 → 把返回的 `widgets` 数组按类型分发渲染（`post_list` 是横滑卡片；`attraction_picker` / `flight_picker` / `hotel_picker` 共用同一套“复选框 + 确认按钮”组件，`max_select` 决定最多能勾几个）→ 再调一次 `/trip` 刷新左侧日程
- 三个“选择型”widget 点“确认选择”之后：把勾中的候选对象原样 `POST /widget-response`，成功后往聊天记录里加一条系统提示（“已确认：xxx”），再刷新一次左侧日程
- 左侧地图**不再是页面加载时初始化一次的单个 `AMap.Map`**（2026-09-11 起改为分帧渲染，跟后端 `trip_map.data.frames` 的结构对应）：每次 `/trip` 刷新，`renderMap()` 先把上一轮所有帧的 `AMap.Map` 实例逐个 `destroy()`，再按 `frames` 数组动态生成对应数量的 `.map-frame` 容器（`#map-frames` 下面纵向排列，每帧一个独立的 220px 高地图 div + “第 N 组 · X 个地点”标签），`drawFrameMap()` 给每帧各自 `new` 一个 `AMap.Map` 实例、只画这一帧自己的 `markers` / `routes`，各帧自动按自己的点位置合适的缩放级别（不用整条行程的极值），互不干扰
- 景点/餐饮用 `AMap.CircleMarker` 按天上色，机票/酒店 marker 固定深灰蓝/棕色（颜色跟 `schedule_widgets.py` 里的常量对应，改了那边记得这边也要改），路线用 `AMap.Polyline` 按天上色，标记点击会弹 `AMap.InfoWindow` 显示地点名（这部分单帧内

部的画法逻辑跟改分帧之前一样，没变)

- `data.unclustered_airports` (离行程主体太远、没并进任何一帧的机场) 渲染成 `#map-frames` 下方一行文字提示 ("距离行程较远, 未画在地图上: xxx"), 不占用地图画布
- 每日行程时间线现在读 `trip.day_timeline` (带颜色) 而不是原始的 `trip.days`, 模块颜色跟地图上同一天的颜色对得上

**现状/限制:** - 按浏览器 session 隔离状态 (见上面"会话隔离"), 但仍是单进程本地 demo, 没有账号登录体系/多进程并发处理, 仅供本地演示, 不要直接这样部署到公网 - 已经端到端联调过完整链路: "推荐+排路线+查机票酒店"发过去 → 右侧出现 `flight_picker / hotel_picker` → 模拟前端 `POST /widget-response` 选中一个航班一个酒店 → 再查 `/trip` 能看到 `flights / hotels` 里多了确认记录 (这一步是直接打 API 验证的, 没配 `AMAP_JS_KEY` 时浏览器里的地图看不到, 但数据链路是通的) - 社区图文面板 (三面板设计里"逛社区帖子墙"那块, 跟聊天框里的 `post_list` 不是一回事) 还没做进网页 - 地图是"重新全量清空再画", 不是增量更新, 行程节点很多的时候会有一点点闪烁, demo 规模不明显

## 待接事项

`main.py / server.py` 已经把所有 Agent + 两套 widgets 串成一条完整链路: 跑 `python orchestrator/main.py` 能看到一次"推荐→规划路线→异常应变→重新渲染行程"的端到端流程; 跑 `python orchestrator/server.py` 打开网页, 能看到"聊天推荐→勾选景点/机票/酒店→确认→左侧地图/时间线/机票酒店面板更新"这条更完整的闭环真的在跑 (已端到端联调验证过)。还剩这几处没做, 都不阻塞基本演示:

1. `ota_hotel_agent` 还是纯占位假数据~~ 酒店这半边已解决 (2026-09-13): 接了 `hotel_tool.py` (道旅 RollingGo, 真实酒店库存/价格); 机票还是纯占位假数据, 没有真实比价来源可接 (调研过携程/去哪儿都是企业商家合作性质, 个人开发者拿不到免费 key)。道旅 RollingGo 的 Partner Center 后台其实有机票产品 (用户 2026-09-13 在自己的后台账号里确认看到过), 但当时状态是"已下线维护", 没能拿到 endpoint/文档——等它恢复了优先去接这个, 不用再重新调研数据源, 直接照 `hotel_tool.py` 的模式 (MCP JSON-RPC over HTTP, 业务数据包两层) 写一个 `flight_tool.py` 大概率能直接抄
2. `route_agent` 只写进同一个占位日期, 没有真正多日期规划~~ **已解决 (2026-09-14)**: 新增 `route_agent.schedule()`, 真正按 `time_budget_days` 新开 `day-N` 多天结构, 到达/结束时间是真推进时间游标算出来的 (不再是 "待定" 占位文字)。旧的 `run(places, ...)` 接口保留不动 (`server.py` 的 `attraction_picker` 确认流程还在用), 只写单一占位日期的行为没变, 两个接口并存
3. `exception_agent` 用整句用户消息去子串匹配地点名字, 很粗糙 (比如"西湖"两个字出现在消息里就命中), 真实版本应该先做实体识别抽出具体地点。cancel 意图 (见第11条) 也是同一套子串匹配, 有同样的粗糙问题
4. 两阶段检索第二阶段 (候选集内部按内容相关性排序) **部分有了替代方案**: `content_agent.nearby_plan()` 的"口碑复核" (真实 POI 查社区评价 + 人格向量匹配排序) 算是对"真实周边 POI 候选"做了一种内容相关性排序, 但这只覆盖 `mode="trip" / mode="nearby"` 扩展出来的 POI 候选, `mode="trip"` 最初的种子帖子 (`store.query_similar_posts()` 直接返回的那批) 依然只有人格相似度排序, 没有二次内容相关性精排, `location_hint` 对种子这部分仍然没有实际使用
5. 反馈闭环 (用户点评行程 → 校准 persona) 还没接: `store.log_history()` 记录和 `persona.apply_feedback()` 校准都写好了, 但没人在 `orchestrate()` 里调用它们
6. `map_tool.py` 目前只查"两点之间", 没做"多点最优顺序"规划——**`route_agent.schedule()` 的 `_greedy_order()` 部分缓解了这个问题** (贪心每次选离当前最近的下一个, 不是真正的最优解, 但至少不再是"按用户/LLM 给的原始顺序排"), `map_tool.py` 本身还是只有两点查询能力没变
7. 三个高德 key 已经申请配置好并真实验证过 (西湖→灵隐寺路线、机场/酒店坐标都是真实高德数据)
8. 讨论过的其他 widget 想法还没做: 反馈评分插件、异常变更确认插件、人格问卷引导插件
9. `单会话全局 state`~~ 已解决: 合并 fellow 分支后改成 `session_store.py` (按浏览器 cookie 隔离, SQLite 存档), 仍然是单进程本地 demo, 没有登录/账号体系, 但至少不同浏览器/不同人打开不会互相覆盖状态了
10. 港澳达人数据库 (229 条, 见 `standardize_hk_macau_data.py` 一节) 已经标准化+导入 Chroma, `verified_local` 优先加权、`city` 硬过滤 (`content_agent.extract_city()` 子串匹配) 都接上了并真实验证过 (问香港只出香港、问澳门只出澳门); 但避坑类帖子的人格向量是中性默认值、`category` (饮食/景点/Tips) 字段还没用来做精细过滤, 这两个属于两阶段检索第二阶段 (第4条) 的范畴
11. `exception_agent` **提案制的"确认后执行"接口还没做~~ 已解决一半 (2026-09-14)**: 新增了 cancel 意图——用户**直接下达删除指令** ("把西湖那站删了") 时, `orchestrate()` 会立刻调 `exception_agent.apply_adjustment()` 真删, 不用二次确认, 因为消息本身就是确认; 已用真实 LLM 分类 + 真删除端到端测过。但 **`exception / check_weather()` 那种"先提案、等用户对提案表态、再执行"的两轮流程还没接**——目前只解决了"用户主动要求删"这一种触发方式, 没解决"系统主动提了个建议, 用户对着建议回答'好/确认'"这种场景 (比如 `check_weather()` 查到真实预警提了建议, 用户回"好, 帮我调整", 现在没有代码能把这

- 句"好"跟"该执行哪个提案"对应起来)。真要做,方向还是 `shared_state` 里存一个 `pending_proposal` 字段记最近一次提案,下一轮识别到"确认"类意图时去执行它
12. **cancel 执行后的"要不要全部重新排"没有对应能力** (2026-09-14, 上一条的直接后续): 删除成功后会固定追问用户"是补新活动还是重新排", 如果用户选"重新排", 目前没有"清空某一天/整个行程重新规划"这个函数——`route_agent / trip_plan` 都只有"追加节点", 没有"清空重排"。用户如果真选了这个选项, 会话会正常走进下一轮 `route / content` 意图识别, 但那两个 Agent 现在的行为是往已有行程上加节点, 不是先清空再排, 效果跟用户预期的"重新排一遍"对不上
  13. **~~两套独立天气数据源~~ 已解决** (2026-09-12): `nearby_sources.weather()` 原来接的是 Open-Meteo, 现在改成统一用 `weather_tool.get_hourly_forecast()` (和风天气逐小时预报, 免费版最多查 240 小时=10 天), 跟 `exception_agent.check_weather()` 用的灾害预警接口共用同一个 key/host。 `nearby_sources.summarize_weather()` 的输入契约也跟着换了 (从 Open-Meteo 那种 `{"hourly": {"time": [...], "temperature_2m": [...], ...}}` 嵌套结构, 改成扁平的 `[{"time": ..., "temperature_c": ..., "rain_probability": ..., "condition_text": ...}]` 列表), 判断"是否有雷暴"从匹配 Open-Meteo 的数值天气码 (`weather_code >= 95`) 改成直接检查和风天气返回的 `condition_text` 里有没有"雷"字, 更直观也不用记一张码表。真实限制: 预报范围从 Open-Meteo 的 16 天缩到了 10 天, 超出范围会走"选定日期不在预报范围内"的降级提示, 不会报错崩溃
  14. **orchestrate() 没有从用户消息里提取日期给 ota\_hotel\_agent.run()** (2026-09-13 接入真实酒店数据后端到端测试时发现, 之前是假数据看不出这个问题): 不管用户说"9月20号"还是别的日期, `booking` 意图命中时传给 `run()` 的 `date_range` 永远是 `None`, 酒店查询永远默认"明天起 2 晚"。真实测试里 LLM 最后组句时诚实地发现了这个落差并跟用户说明 (防幻觉提示词生效了), 但功能上确实没做到"按用户说的日期查"。修法待定, 两个方向: 简单加一段日期正则/关键词提取; 或者扩展 `classify_intent` 顺带做结构化实体抽取 (日期/星级/预算这些一起解决), 需要先定思路 (`orchestrator_agent._extract_day_count()` 后来给"几天"这个更简单的场景加了正则提取, 见第2条, 但完整日期/日期区间的提取还是没做)
  15. **~~ web/nearby.js / web/index.html 没跟着达人 Agent 重构改~~ 已解决 (2026-09-15)**: 走的是上面提到的第一个方向——`web/nearby.js` 整个删掉, `web/index.html` 里的独立附近游表单 (城市/起点/日期/时长字段 + 提交按钮)、专属地图渲染代码 (`drawNearbyMap / renderNearby / restoreNearby`)、相关 CSS 全部拿掉, 左侧面板恢复成单纯的"实时日程"展示。`server.py` 的 `/nearby-plan / /nearby-weather / /nearby.js` 三个路由也从"留着但报错的 501 占位"改成彻底移除 (路由表里已不存在, 请求直接 404)。附近游现在完全走 `POST /chat` 自然语言 ("从大三巴出发逛3小时"), 排班结果写进主 `trip_plan`, `GET /trip` 自然带出来, 前端不用维护第二套请求/渲染逻辑。真实浏览器验证过页面渲染干净, `tests/test_web_sessions.py` 同步更新 (`test_nearby_endpoints_removed`, 断言 404)
  16. **~~ map\_tool.route\_between() 对港澳场景两个挨得很近的真实地点偶尔查询失败~~ 已解决 (2026-09-15), 顺带修了地图坐标丢失的问题**: 真实 demo 走查时发现两个相关问题——(a) 这一条记录的路线查询偶发失败; (b) `GET /trip` 的地图组件 (`schedule_widgets.build_trip_map_widget()`) 有时会把行程拆成好几张地图、点还落在离港澳很远的地方。根因是同一个: `trip_plan.add_stop()` 存节点时只存地点名字字符串, 不存坐标——虽然 `content_agent.py` 早就用 `nearby_sources.find_origin() / nearby()` 查过真实、可靠的坐标了, 但 `route_agent._place_day()` 写节点时把这些坐标扔了; 后续不管是 `route_agent._real_leg()` 算交通时间, 还是 `schedule_widgets.build_trip_map_widget()` 画地图, 都得拿地点名字重新走一次 `map_tool.route_between() / map_tool.geocode()` ——而这俩内部用的 `geocode/geo` 接口正是这个 session 已经证实对港澳不可靠的那个 (同名地点解析到千里之外)。修法: ① `route_agent._place_day()` 写节点时把 `lng / lat` 一起存进去 (`trip_plan.add_stop()` 本来就支持 `**extra` 透传, 不用改 `trip_plan.py`); ② `route_agent._real_leg() / schedule_widgets._route_leg_coords()` 坐标已知时改用 `nearby_sources.route(a, b, mode)` (早就存在但之前没人用的坐标直查接口, 直接传经纬度算路线, 不再地理编码); ③ `build_trip_map_widget()` 拼地图标记时优先用节点上存的坐标, 没有才退回按名字查。真实验证: `mode="nearby"` 排一次班, 全程零次 `map_tool.geocode()` 调用、地图只生成 1 个框、`geocode_failures / route_failures` 都是空列表。老的 `route_agent.run()` (`server.py` 的 `attraction_picker` 确认流程还在用) 当时保留了原来的按名字查询兜底行为——**但这条路径本身就是不可靠地理编码的活跃触发点, 不是"没有回归"就完事了**: 真实测试确认用户点"确认选择"之后, 地图上出现过"上海""新疆"这种离谱定位。根因是 `server.py` 的 `apply_selection()` 把选中候选压缩成只剩 `place` 字符串 (`item.get("place")`), `content_agent` 早就查好的 `lng / lat` 被丢弃, `route_agent.run()` 只能重新按名字地理编码。同一次修复 (2026-09-15 第二轮) 补上了这条路径: `apply_selection()` 改传完整候选字典, `route_agent.run()` 坐标已知时改用 `_real_leg()` (跟 `schedule()` 共用同一个坐标直查逻辑), 没坐标才退回老的按名字查——`run()` 的 `__main__` 自测块等纯字符串调用方不受影响。真实验证: 传真实候选字典, 全程零次 `map_tool.geocode()` 调用
  17. **content\_agent.run() (mode="trip") 种子选择没有按地点去重** (2026-09-15 真实 demo 演示时发现): `store.query_similar_posts()` 按人格相似度取 `top_k5` 候选, 只过滤掉 `category=="Tips"`, 没有任何"同一地点只留一条"的约束——两条不同的社区帖子都在夸同一家店 (真实测过"叠记咖喱美食") 完全可能同时挤进种子名额, 变成同一个地方被推荐两次, LLM 组句时自己都发现了这个重复但没法从候选层面修掉。原有的 `seen_places` 去重 (`content_agent.py` 里种子扩展周边

那段) 只防"种子 vs 周边扩展"/"周边扩展 vs 周边扩展"重复, 没防"种子 vs 种子"重复。已解决": 新增

`content_agent._dedupe_by_place()`, 在 `Tips` 过滤之前先按 `place` 去重 ( `fetches` 已经按 `similarity_score` 降序排好, 遇到的第一条就是相似度最高那条, 后面同地点的直接跳过)。真实验证: 同样的人格/城市查询, 75 条推荐去重前后地点数一致 ( `len(places) == len(set(places))` ), 不再有重复。这一条跟第16条 (坐标丢失) 是同一轮真实 demo 演示发现的, 但是两个独立根因, 分开提交

18. `route_agent.schedule()` **mode="trip" 排班改成"时段骨架", 不再是纯最近邻贪心+一天11小时预算** (2026-09-15): 真实 demo 演示"帮我推荐一个澳门3日游"时用户发现两个结构性问题——(a) 每天推荐清一色是美食, 没有景点; (b) 隔得很远的候选被排进同一天。根因: ① `content_agent.run()` 的种子选择纯按人格相似度取 `top_k`, 没有类别配额, 人格标签偏"美食探店"时种子经常全是"饮食"类, 每个美食种子扩展出的"周边真实 POI"大概率也还是餐厅, 偏差被放大; ② `route_agent._greedy_order()` 是纯最近邻贪心, 没有单段距离/耗时上限, 一天的时间预算是粗粒度的 11 小时, 种子分散在城市不同区域时, 某天贪心选完一个种子附近的候选后可能去抓另一个种子附近的候选, 只要没超预算就会被排进同一天, 跳跃很远。

**已解决**, 用户提出的架构方向: 编排 Agent (这里落在 `route_agent.py`, 符合排时间接口归 `route agent` 的既定分工) 先定好每天的"时段骨架", 路线 Agent 往里面填候选, 槽位类型/时间窗口天然保证类别配比和距离约束: - 新增 `_SLOT_TEMPLATE`: 上午(09:00-12:00)/下午(14:00-17:30, 可选)/晚上(19:30-21:30, 可选)三个景点槽位 + 午餐(12:00-13:30)/晚餐(18:00-19:30)两个餐饮槽位, 上午/午餐/晚餐是必选锚点。槽位之间是硬楼层 ( `cursor=max(上一段结束时间, 这个槽位start)` ), 不会因为上一段提前结束就提前开始下一段, 刻意模拟真实旅游的节奏感。- 候选按新增的 `_candidate_role()` (同时查"食"/"餐"两个字, 覆盖社区帖子"饮食"分类和高德 POI"餐饮服务"分类两种来源——顺手修了 `_stay_minutes()` 原来只看"食"字、对高德餐饮 POI 大概率判不出来的隐藏 bug) 分成景点/餐饮两个候选池, `_fill_day_skeleton()` 每个槽位从对应池子挑离当前位置最近、真实交通+停留时长能在槽位窗口内完成的候选 (挑不到试下一个, 最多试 3 个就放弃); 景点槽位排完主候选后, 槽位还剩至少 45 分钟且候选池里有 800 米内的同角色候选, 会再加塞一个 (不算锚点); 全天节点数不超过 `_MAX_STOPS_PER_DAY=6`。- 一天排完 (还没真正写进 `trip_plan`) 调一次 `_llm_review_day()` ( `MODEL_LIGHT`, 不是每加一站调一次——这个 demo 串行真实 API 调用已经要几分钟, 逐候选调模型会让延迟更差) 做"是否影响体验"的软性复核, 只能拿掉加塞/可选槽位的候选, 三个必选锚点不在可拿掉范围内 (结构性保证的保底), LLM 调用失败/解析失败一律降级成"不拿掉", 不拖垮排班。- `content_agent.run(mode="trip")` 种子挑选新增 `_pick_seeds_with_category_balance()`: 优先保证至少 1 条"景点"类种子, 不够就照常按相似度回退, 不强求。

范围只改 `mode="trip"`; `mode="nearby"` ("从大三巴出发逛3小时") 现状良好, 继续用原来的 `_greedy_order()` / `_place_day()`, 没动。顺手修了一个不相关但相关代码路径里发现的显示 bug: `_place_day()` /新排班代码之前不管景点还是餐饮, 写进 `trip_plan` 的 `type` 字段都硬编码成 "attraction", 前端 ( `web/index.html` ) 靠这个字段区分 🍽️ 餐饮图标和 📍 景点图标, 餐饮类之前全部被错误显示成景点——现在按候选的真实角色写 `type / meal_type`。

真实验证: 三天真实数据排班, 有真实候选的那天正确混出"2个景点+2顿饭"结构 ( `type / meal_type` 字段正确), 单段真实交通距离都在几百米以内, 没有远距离跳跃; 候选池数据本身覆盖不到的天 (比如种子全是同一片区域的美食、附近查不到景点 POI) 会诚实地只排出能排的 (比如只有午餐晚餐), 不会为了凑数硬塞不合理的候选——这是"229条社区帖子覆盖有限"这个已知数据局限的自然体现, 不是算法 bug。地图 widget 复测仍然是 0 次 `map_tool.geocode()` 调用。**已知局限**: 槽位时间是写死的模板, 没有根据用户说的"轻松点"/"紧凑点"调节节奏; 候选池天生只有美食 (人格向量/该城市数据分布导致一条"景点"都检索不到) 时, 景点槽位会一直空着, `_pick_seeds_with_category_balance()` 只能在候选池里"确实有景点"的前提下起作用, 不能凭空生成候选

1. `ota_hotel_agent` **酒店搜索不管已排行程的位置, 无差别搜整个城市** (2026-09-15 真实 demo 演示时发现): 用户问完 3 日游行程后接着问"帮我订酒店", 编排 Agent 兜底的自由文本组句"聪明地"看着 `trip_plan` 里的行程位置建议住哪个区域 (比如"你的行程都在新马路一带, 别住氹仔")——但那只是模型看 JSON context 自由发挥, `ota_hotel_agent._search_real_hotels()` 真正调 `hotel_tool.search_hotels()` 时一直传 `place=city, place_type="城市"` ( `orchestrator_agent.py` 调用时也是 `location=None`, 完全没把已排的行程位置传进去), 从来没按行程收窄过——嘴上说的和真正会出现在 `hotel_picker` 卡片里让用户选的候选不是一回事, 用户点卡片照样能选中文字里劝退的氹仔酒店。真实验证过: 同样查询条件下, `place="澳门"`, `place_type="城市"` 的结果里有氹仔的"英伦游艇会酒店", `place="大三巴"`, `place_type="景点"` 的结果里没有。

**已解决**: 新增 `ota_hotel_agent._anchor_place_from_trip()`, 从 `shared_state["trip_plan"]` 里取最早一天第一个带坐标的站当"锚点" ( `{ "place": "lat": "lng": "lat": }`, 坐标是排班时就存好的真实坐标, 见第18条), `_search_real_hotels()` 有锚点就传 `place=锚点地点, place_type="景点"` (没有就退回原来的 `place=city, place_type="城市"`)。踩了一个新坑: 锚点不够出名时 RollingGo 会整个匹配错地方——真实测过"中西药局旧址" (澳门一处不太热门的历史建筑) 被匹配到美国圣路易斯, 坐标直接跑去密苏里州。加了一道合理性校验: 用锚点已知的真实坐标跟返回的酒店坐标算距离, 超过 `_ANCHOR_SANITY_KM=50` 公里就当整批匹配失败, 自动退回城市级搜索重查一次, 不把跑偏的结果直接返回给用户。**已知局限**: 只有锚点本身是知名地标 (比如"大三巴") 时才能可靠收窄, 社区帖子/高德 POI 里大量不那么出名的地点 (历史建筑、小吃店) 大概率触发合理性校验退回城市级搜索——退回去之后行

为等同于改动前，不会更差，只是没能享受到收窄的好处；不强求每次都收窄成功，符合项目一贯“能收窄就收窄，收窄失败就老实退回”的降级哲学。

- 聊天框把编排 Agent 的 Markdown 回复当纯文本显示，## / \*\* / | 表格 | 这些符号原样露在外面** (2026-09-15 用户反馈)：web/index.html 的 addMessage() 一直是 `div.textContent = content`，编排 Agent 通用组句 (orchestrator\_agent.py 里没命中 restaurant / cancel 时走的 LLM 自由组句分支) 经常输出结构化的 Markdown (标题/加粗/表格/列表)，用户看到的是整段带着原始符号、没有任何格式的文字。**已解决**：引入 `marked.js` (cdnjs CDN, v9.1.6, 纯前端库，不需要改后端)，addMessage() 对 `role===“assistant”` 的字符串内容改成 `marked.parse()` 转出 HTML 再塞 `innerHTML` (user / system 角色继续用 `textContent`，用户自己打的字没必要也不该被当 HTML 渲染)；sendMessage() 里单独处理 `pendingEl` 的那处直接赋值也一并改了。`marked` 没加载成功 (比如断网、CDN 被拦) 会优雅退化回纯文本，不让聊天框崩掉。配了一套 `.msg.assistant` 下的标题/表格/列表/引用样式，不是用浏览器默认样式硬堆在聊天气泡里。真实验证：用户之前反馈的那段酒店推荐 Markdown 原文重新走一遍渲染，表格/加粗/分级标题/列表全部正常显示。
- 编排 Agent 组句回复的行程跟 trip\_plan 实际排定的不一样** (2026-09-15 用户反馈：文字里 Day2 的行程跟左侧面板实际排的 Day1 对不上)：orchestrate() 通用组句分支 (else 兜底那段) 把 `shared_state["trip_plan"]` 原始链表结构 (`{“head_id”:, “nodes”:{id:{..., “next_id”:}}}`) 连同 `results.content.recommendations` 全量候选池 (真实测过 75 条，带 `community_reviews / images / tags` 等重字段) 一起塞进 context，模型很难正确按链表顺序转述每天的行程，加上候选池信息量太大，容易把“候选”和“已经排定的行程”搞混，自己把候选重新编排一遍，跟 `route_agent.schedule()` 真实排的顺序/天数对不上。**已解决**：trip\_plan 改传 `trip_plan.render()` 之后的整理版 (`{“days”:[{“date”:, “stops”:[有序数组]}, ...]}`)，recommendations 新增 `_trim_recommendations_for_reply()` 精简成只剩 `place / category / source`；system prompt 里明确加了一句“trip\_plan.days 是已经真实排定的行程，必须按它如实转述，不能自己重新编排；recommendations 里没排进 trip\_plan.days 的地点只能提成备选”。真实验证：重新跑一次“9.15-9.18 澳门3天”的真实请求，回复文本的三天地点/顺序/时间跟 `trip_plan.render()` 逐字对上，未排入的候选老实标成“备选地点 (还没排进行程)”。
- 排时间的触发点从“意图分类猜中了 route”改成“用户从 attraction\_picker 选完确认”** (2026-09-15)：真实测试发现同一句“帮我推荐一下怎么玩”排不排班完全取决于 `classify_intent()` (一次真实 LLM 调用) 有没有把这句话分进“route”意图，同一句话不同轮次分类结果可能不一样，导致“这次有行程表、下次没有”；而且原来自动排班用的是 `content_agent` 算出来的全量候选池 (真实测过 75 条)，用户还没表达任何偏好，算法就自己把整个行程定了。**已解决**：`orchestrate()` 不再有 `content + route` 意图 → 自动排 `mode="trip"` 这条分支 (`mode="nearby"` 不受影响，“从大三巴出发逛3小时”这种直接行动请求命中就照常立刻排，它没有“先给候选、等用户挑”这一步)；`classify_intent() / _INTENT_SYSTEM_PROMPT` 里 `route` 意图整个删除 (改动后没人用了)；server.py 的 `apply_selection()` 的 `attraction_picker` 分支从老接口 `route_agent.run()` 改调真正的骨架排班 `route_agent.schedule()` (`mode="trip", time_budget_days=1`) ——用户选完确认才是第一次真正排班的时刻，且用的是用户自己选的候选，不是算法全池子自动决定的。`attraction_picker` 选择上限从 3 放宽到 6 (`widgets.py`，对齐 `_SLOT_TEMPLATE` 一天最多 6 个节点的上限)，候选池 `pool_size` 从 6 调到 10 (给用户留出真正能挑的空间)。多天行程靠多轮“推荐→选→确认”自然累积 (`schedule()` 本来就会找第一个没用过的 `day-N` 续排，不需要新状态)。老接口 `route_agent.run() / _estimate_transport()` 改完之后彻底没有调用方了，直接删除 (连同 `orchestrator_agent._extract_day_count()`，唯一调用点被删掉后也成了死代码)。真实验证：POST /widget-response 走 `attraction_picker` 确认真实候选，确认 `route_agent.schedule()` 被调用、`trip_plan` 首次被填入、零次 `map_tool.geocode()` 调用。
- 达人 Agent 新增候选分类质检，标错了顺手改数据库** (2026-09-15，用户提出)：`route_agent._candidate_role()` 靠 `category` 字段里有没有“食”/“餐”字“分餐饮/景点”，本质是信任上游数据——社区帖子的 `category` 是人工录入的，可能真的标错 (比如一家餐厅被手误标成“景点”)，标错一条就可能把餐厅排进景点槽位，绕开骨架排班“类别配比”这个结构性保证。**已解决**：`content_agent.py` 新增 `_verify_and_fix_categories()`，候选提交给编排 Agent 之前，一批候选一次性打包成一次 `MODEL_LIGHT` 调用做质检 (不是逐条调用，控制延迟)，发现标错的就地修正、带 `post_id` 的 (社区帖子来源) 顺手调新增的 `store.update_post_category()` 写回数据库——验证过 Chroma 的 `update()` 是合并式更新，只改 `category` 这一个字段，`place / caption / tags` 等其他字段原样保留，不会因为一次分类修正把整条帖子的其他数据冲掉。高德 POI 来源没有持久化数据可写，只改这次返回结果的内存字段。LLM 调用/解析失败就什么都不改，不影响 `content_agent.run()` 主流程。真实验证：229 条真实数据跑一次没有触发误报/崩溃 (这批数据本身没查出真实标错的，属于正常情况，质检逻辑本身用真实 Chroma 读写单独验证过)。
- 酒店锚点改成“多天行程末站重心” + 已订酒店的距离复核** (2026-09-15，用户提出)：第16条加的 `ota_hotel_agent._anchor_place_from_trip()` 原来只取“最早一天第一个带坐标的站”当酒店搜索锚点——多天行程时只照顾了

第一天，比如 Day3 行程跑去冰仔，酒店搜索完全没考虑到。用户指出订酒店应该看**每天最后一站**（晚上回酒店睡觉，最后一站离酒店远不远才是真正影响住宿体验的），且已经订好的酒店只要不是特别远就不用换。**已解决**：

- 新增 `_days_last_stops()`：取每天最后一个带坐标的节点（不是第一个）。
- `_anchor_place_from_trip()` 改成取**所有天数最后一站坐标的重心（平均值）**，不再只看某一天；RollingGo 的 `place` 参数要真实地名，用离重心最近的那一站地点名字做文字检索，但坐标本身用重心点（后续合理性校验更准确地反映“整租行程”）。
- 新增 `_existing_hotel_distance_check()`：已经订好酒店（`trip_plan.hotels` 非空）时，检查现有酒店离每天最后一站够不够近（`_HOTEL_CONVENIENCE_KM=5` 公里内不用管），超出的天数只在回复文字里提醒  
(`result["hotel_distance_reminder"]`)，**不自动重新搜索/推荐换酒店**——酒店是用户已经确认的真实决定，系统不该擅自建议换。
- `run()` 里：已经订了酒店就不再发起新的酒店搜索（`booking` 意图每次命中都重新搜一遍候选会显得像是在暗示用户换酒店，不合理），改成只做距离复核。
- **顺带修的前置缺口**：`trip_plan.add_hotel()` 存进去的酒店记录之前没存坐标（`hotel_tool.search_hotels()` 明明有 `lat / lng`，只是 `_search_real_hotels()` 构造候选字典时没往下传，`server.py` 的 `apply_selection()` 也没存）——这是这次对话反复出现的同一类“坐标算出来了但传递链路中间丢了”问题，这次一并接上（`_search_real_hotels()` 候选带上 `lng / lat`，`apply_selection()` 的 `hotel_picker` 分支存进 `trip_plan.add_hotel()`），不然“已订酒店距离复核”这个新功能根本没有坐标可用。

真实验证：两天真实行程（`content_agent.run()` + `route_agent.schedule(time_budget_days=2)`），`_anchor_place_from_trip()` 返回的坐标跟两天末站真实坐标的算术平均值完全一致；模拟一个真实离行程很远的已订酒店（冰仔坐标），确认 `run()` 没有发起新的酒店搜索（`_search_real_hotels` 断言未调用）、`hotel_distance_reminder` 正确点名了两天行程终点都偏远。

1. **候选筛选加硬性类别配额**：景点 $\geq 2 \times$ 天数、早餐 $\geq$ 天数、午晚餐 $\geq 2 \times$ 天数，早餐/午晚餐分开管理（2026-09-16，用户提出）：用户问“一开始选的时候会区分景点和美食吗”——现状只有一个软性兜底（种子里至少 1 条“景点”，剩下纯按相似度），周边真实 POI 扩展完全没有类别配额意识；用户建议加硬性配额，且要把“美食”细分成早餐/午晚餐（保证早餐/午晚餐候选池数量分别  $\geq$  天数/ $\geq 2 \times$ 天数，才能保证  $n$  天不撞同一家店）。**已解决**，贯穿 `content_agent.py`（选）和 `route_agent.py`（排）两个 Agent：
  - `content_agent.py` 恢复天数提取（`_extract_day_count()`），逻辑跟上上个提交删掉的 `orchestrator_agent` 版本一样，这次只用来决定该捞多少候选，不触发自动排班；候选池扩大到按天数缩放（`min(20 \times \text{天数}, 150)`）；新增 `_pick_seeds_with_quotas()` 替换原来“至少 1 个景点”的软性兜底，按  $2 \times \text{天数} / \text{天数} / 2 \times \text{天数}$  强制配额挑种子。
  - 新增分类质检 `_classify_categories()`（原 `_verify_and_fix_categories()` 升级，二选一“饮食/景点”改成三选一“景点/早餐/午晚餐”，社区帖子/高德 POI 都要走一遍）——**提示词踩了一个坑**：真实测过用“如果实在看不出来就保留原分类”这种宽松措辞，一批 34 条候选里 15 条会被模型偷懒原样保留成笼统的“饮食”，改成“只要不是精确的三选一标签就必须归类，餐饮类判断不出早晚就默认午晚餐”这种强制措辞后，同一批数据、以及真实 3 天行程里全部 290 条候选（243 条来自周边高德 POI 扩展）都被归类干净，零遗留。
  - `route_agent.py` 的 `_SLOT_TEMPLATE` 加早餐槽位（08:00-09:00，必选锚点），槽位标识从“attraction/meal”二选一角色改成“attraction/breakfast/lunch\_dinner”三选一池子（`_pool_for_candidate()`，候选没被分类过时兜底默认归 `lunch_dinner` 不归 `breakfast`）；`_fill_day_skeleton()` / `schedule()` 相应改成管理三个候选池；`_MAX_STOPS_PER_DAY` 从 6 调到 7（6 个槽位+1 个景点加塞）。
  - **顺带解决另一个用户提出的缺口**：“传回去的数据会有推荐指数吗”——发现数据其实已经有 `similarity_score`（社区帖子）/ `persona_match_score`（周边 POI）两个打分字段，但 `route_agent.py` 从头到尾没读过。统一成 `recommendation_score` 字段，传进 `_llm_review_day()` 的复核 payload，让“删减”判断有分可依（同等情况优先拿掉分数低的）——`pick_one()` 选点本身还是按距离/时间可行性，跟“推不推荐”是两回事，不混在一起。

真实验证：真实 3 天行程（`content_agent.run(location_hint="推荐一个3天的行程")` + `route_agent.schedule(time_budget_days=3)`），290 条候选全部归类成干净的三选一标签（景点43/早餐6/午晚餐241）、全部带 `recommendation_score`；排完的 3 天共 17 站，`meal_type` 正确区分早餐/午餐/晚餐三档，**17 个地点互不重复**（`len(set(all_places)) == 17`）。

1. **新增行程偏好确认**：搜索前先问酒店/机票偏好，跟 **persona 已有维度交叉引用**（2026-09-16，用户提出）：用户希望编排 Agent 在真正调用达人 Agent 搜索之前，先确认旅行模式、酒店需求、机票倾向、旅行节奏，做到更精准推荐；并问能不能跟 `persona` 系统交叉引用。**查过 `persona.py` 确认**：四个维度里三个已经存在——“旅行节奏”= `pace_score`、“度假 vs 出差”= `scenario`、“逛吃美食 vs 景点打卡”= `interest_theme` 标签，都不需要重新问/重新存。真正没被覆盖的只有**酒店偏好**（周边图方便 vs 单独选一家好酒店当度假体验，不等于 `budget_score`）和**机票优先级**（省钱/折衷/极致体验）。**已解决**：

- 新增 `orchestrator/trip_preferences.py` : `TripPreferences` 数据类只存这两个新字段, `describe_for_persona()` 从 `persona` 读"节奏"/"模式"给调用方用, 不重复存一份 (避免同一个概念存两处、改一个忘了改另一个)。
- `orchestrator_agent.py` : `new_shared_state()` 加 `trip_preferences / pending_trip_request` (暂存"问偏好之前用户真正想说的那句话") 两个字段; `orchestrate()` 开头新增确认流程——`content` 意图命中且 `nearby` 未命中 (即 `mode="trip"` 常规推荐场景) 时, 若偏好没问完, **一次性**列出所有缺失维度的追问 (用户已用 `AskUserQuestion` 确认过是一次问完不是分好几轮), 把原始请求存进 `pending_trip_request`, 短路返回; 下一轮如果 `pending_trip_request` 非空, 就把这轮消息当回答解析 (`_parse_preference_answer()`), 一次 `MODEL_LIGHT` 调用, 用户说"继续"/解析不出来的字段用默认值兜底, 不会卡住对话), 然后取出暂存的原始请求继续走正常分发。`mode="nearby"` ("从大三巴出发逛3小时") 不需要先确认, 跟它不走 `attraction_picker` 确认流程是同一个道理。
- `ota_hotel_agent.py` : `hotel_preference="度假酒店"` 时不走 `_anchor_place_from_trip()` 的锚点收窄 (用户不介意离行程远), 改成城市级搜索 + 提高星级门槛 (`_search_real_hotels()` 新增 `star_min` 参数透传给 `hotel_tool.search_hotels()`, 那边接口本来就支持, 之前没接上); `flight_priority="省钱"` 时把假机票候选按价格排序排——**如说明局限**: 机票是假数据 (`_generate_beijing_flights()`), `flight_priority` 目前只能影响候选排序, 不能真正筛选时刻/舱位。

真实验证: 真实两轮对话, 第一轮"推荐一个3天的澳门行程"触发确认 (`community_panel` 为空, `pending_trip_request` 正确存住原始请求); 第二轮"周边方便就行, 机票随便"正确解析出 `{"hotel_preference": "周边", "flight_priority": "折衷"}`, 用暂存的原始请求真正触发了搜索 (290 条候选), 回复文本里正确引用了 `persona` 的 `interest_theme / pace_score` ("自然风光、美食探店、家庭出游、节奏偏慢")。

- 1. 周边 POI 扩展没有筛选/截断, 候选量爆炸** (2026-09-17, 用户反馈"周边POI也太多了"): 上一条验证时"3天澳门行程"实测出了 290 条候选, 根因是 `content_agent._nearby_plan()` 对 `nearby_sources.nearby()` (AMap 同城 POI 检索, 一次能回几十条) 的结果完全不截断——`run()` 里 `mode="trip"` 场景对每个种子 (第25条硬配额下, 3 天行程最多能有  $2 \times 3 + 3 + 2 \times 3 = 15$  个种子) 都各自调一次 `_nearby_plan()`, 汇总去重后自然是几百条量级, 两轮 `_classify_categories()` 的 LLM payload 也跟着臃肿。用户建议参考已删除的 fellow 分支 `nearby_planner.py` (`git show <deletion-commit>~1:orchestrator/nearby_planner.py` 找回来看的)——那边的做法是先用一次 LLM 调用把候选"从资料中选出最多6个地点" (`rank_candidates()`), 再 `pool = ranked[:8]` 硬截断到 8 个, 才进贪心排线路。**已解决**: `_nearby_plan()` 新增 `limit` 参数 (默认 `_NEARBY_PER_CALL_CAP=8`, 直接对齐 fellow 分支当年截断到 8 的量), 按已有的 `persona_match_score / recommendation_score` (口碑复核算出来的排序分) 排序后截断——跟 fellow 分支的目的 (别把一次周边查询的全部结果都塞给下游), 但不需要 fellow 那一步额外的 LLM 排序调用: 那边当年没有现成的排序依据, 只能靠 LLM 挑; 现在口碑复核已经算出了结构化分数, 直接按分数截断效果等价, 还省一次 LLM 调用和一次可能的解析失败点。真实验证: 同样"我想在9.15从北京出发去澳门玩3天"的真实请求, 候选总数从 290 降到 58~66 (景点/午晚餐类别配额依然满足, 早餐类目前为 0 条——查证是社区帖子库里澳门本来就没有早餐类型的帖子, 第25条配额"尽力而为不强求"的既有降级行为, 跟这次截断改动无关, 不是新引入的回归)。
- 2. attraction\_picker 确认排班只喂用户勾选的那几个候选, 达人 Agent 找到的其它候选被晾在一边** (2026-09-17, 用户提出"难道编排 Agent 不应该把所有达人选出来的给路线 Agent 吗, 只是用户选了的优先级更高"): 第22条把排班时间的触发点从"意图分类"改成"用户在 `attraction_picker` 选完确认"之后, `server.py` 的 `apply_selection()` 一直是 `route_agent.schedule(state, picks, ...)`——`picks` 只是用户在候选卡片里勾的那几个 (最多 6 个, 而且候选卡片本身就只展示 `widgets.build_attraction_picker_widget()` 精选出的 `pool_size=10` 个, 不是达人 Agent 的全量候选池)。`route_agent.schedule()` 排班时只看得到这几个候选, 早餐/午晚餐/用户没勾的其它景点槽位完全没有候选可用, 只能空着——不是"用户选的优先", 是"用户没选的直接不存在"。**已解决**:
  - `orchestrator_agent.py` : `orchestrate()` 生成 `attraction_picker` widget 的同时, 把达人 Agent 给的全量 `candidates` 存进 `shared_state["last_content_candidates"]` (`new_shared_state()` 也加上这个默认字段)——`attraction_picker` 的确认是另一次独立的 HTTP 请求 (`POST /widget-response`), 到那时候本轮 `candidates` 早就不在调用栈里了, 必须存进 `session` 状态才能跨请求拿到。
  - `route_agent.schedule()` 新增 `priority_places` 参数 (候选 "place" 字段的集合), 透传给 `_fill_day_skeleton()`: 槽位内候选试探顺序从纯按距离排序, 改成 (是否命中 `priority_places`, 距离) 二级排序——命中的候选在同池子里永远排最前面优先尝试 (排不进当前槽位再轮到后面按距离排的其它候选), 不是"只排这些"。
  - `server.py` 的 `apply_selection()`: `attraction_picker` 分支改成把 `state["last_content_candidates"]` (全量候选池, 取不到就退回只用 `picks`, 不炸) 交给 `schedule()`, 用户勾选的 `picks` 转成 `priority_places` 传进去; 衔接回复 (见下一条) 改成检查用户勾选的地点有没有真的排进去, 不是检查全量池子的剩余数量 (全量池子本来就有几十条, 绝大多数因为一天排班槽位有限进 `unscheduled` 是正常现象, 不是"没排上")。

3. **attraction\_picker / flight\_picker / hotel\_picker 确认之后聊天框没有任何反应** (2026-09-17, 用户反馈“能不能做完交互后加点衔接提示”) : `POST /widget-response` 一直只返回 `{"status":"ok"}`, `trip_plan` /地图面板悄悄更新了, 但聊天框只有前端自己拼的一句“已确认: 地点1、地点2”, 用户看不出排班/订票订房这些后台动作到底成没成功。已解决: `apply_selection()` 现在返回一句话总结实际发生的事——景点排完之后说排进了哪天, 用户勾的地点有没有全部排上、没排上的具体是哪几个; 机票/酒店订完之后说订好了什么, 酒店额外带上 `_existing_hotel_distance_check()` 的距离提醒 (原来只有聊天流程接了这个提醒, `widget` 流程没接)。这句话存进 `state["messages"]` (下次 `/session` 刷新还在) 也直接回传给前端这次响应用, 前端 `renderPicker()` 确认成功后除了原来的“已确认: ...”系统提示, 多显示一条助手消息气泡。
4. **\_extract\_day\_count() 日期区间算成了“晚数”, 少算一天** (2026-09-17, 用户指出“9.15到9.18其实是4天”) : 第27条刚加的日期区间兜底用 (结束日期-开始日期).days 算天数, 9.15到9.18算出来是3——这是两个日期之间的“晚数”, 中文口语“4天3晚”里的“天数”要把首尾两天都算进去, 得在晚数基础上 +1。已解决: 日期差 +1 再封顶 14 天; 同时把这次改动跟第26条“显式 N 天 优先于日期区间兜底”的顺序验证了一遍, 没有互相影响。
5. **route\_agent.schedule() 排班没有地理聚类, 也不知道机票时间** (2026-09-17, 用户提出两点: ① “排路线也该把离得比较近或者顺路的排一天”, 建议三步法——先按 `recommendation_score` 选排名靠前的候选, 再按距离/通勤时间聚类分天, 最后照旧塞加塞候选; ② 机票的到达/离开时间要嵌入排班, 落地当天/离开当天不能套用完整一天的时段骨架, 且“用户选中的景点如果被冲掉了要挪到别的天, 按距离看哪天最合适”)。原来 `_fill_day_skeleton()` 是全程共享一个大候选池, 按天贪心从槽位第一个开始排, 同一天的候选之间没有“地理上该扎堆”的约束, 也完全不知道有没有订机票、机票几点起降。已解决, `route_agent.py` 新增/改动一批函数:
- `_centroid() / _greedy_geo_cluster() / _assign_by_proximity()` : 三步法的第①②步。景点候选先按 `recommendation_score` 排名当“种子”, 贪心聚成 `time_budget_days` 组 (同一天的候选地理上尽量扎堆, 不是天南地北乱跳); 早餐/午晚餐按“离哪天的景点聚类重心最近”分天, 保证顺路, 不会“早餐城北、景点城南”。`priority_places` (用户在 `attraction_picker` 里勾选的地点) 会优先当聚类种子, 正常排队还是没能分到任何一天 (比如所有天容量加起来都不够, 或者被机票时间挤没了槽位) 会有兜底: 硬塞进离它最近、已经有候选的那一天, 哪怕因此超出这天原定容量——用户明确选的地点不能因为排班算法的配额算不过来就消失在 `unscheduled` 里。
  - `_flight_arrival_and_departure()` : 从 `trip_plan.flights` 里找“飞到本次城市”的到达时间、“从本次城市飞走”的起飞时间 (demo 机票数据没有专门的 `direction` 字段, 用 `to_ / from_` 跟城市名比对判断方向, 够用不追求精确)。
  - `_clip_slot_template() / _day_slot_templates()` : 把抵达当天的槽位窗口裁到不早于“到达时间+2小时缓冲” (过关取行李通勤), 离开当天裁到不晚于“起飞时间-3小时缓冲” (去机场值机安检)——是裁窗口不是整槽丢弃; 落地10:10、缓冲到12:10, “午餐”槽位名义上12:00开始、窗口到13:30, 裁完变成12:10-13:30还留得住, 不会因为名义开始时间比这天能动的时间早10分钟就整个作废。
  - `_fill_day_skeleton()` 新增 `slot_template` 参数 (默认还是完整的 `_SLOT_TEMPLATE`), `schedule()` 给抵达/离开当天传裁剪过的版本, 中间的天不受影响; 这天砍到一个槽位都不剩 (比如当天来回) 就原样不排, 不报错。
  - `schedule()` 把景点/早餐/午晚餐三个池子的“每天容量”从写死的常数改成按 `_day_slot_templates()` 算出来的每天实际槽位数——抵达/离开当天容量自然变小, 聚类那一步提前知道这天装不下多少, 不会先分多了再被槽位挡回去。
- 真实验证: 已订机票 20:00 落地 (缓冲后当天景点容量变 0), 用户在 `attraction_picker` 里勾的一个景点排 2 天时被正确挪到了 day-2 (没有消失在 `unscheduled` 里); 两组地理上隔得很开的景点候选 (模拟“城中”/“路环”) 排 2 天行程时确实各自成组, 没有拆开混进同一天。
6. **机票候选带 date 字段, 存进 trip\_plan.flights 时手动拼字段漏掉了, 前端也没渲染** (2026-09-17, 用户反馈“机票展示这一栏没有日期这不合理”) : `ota_hotel_agent.run()` 生成的机票候选其实一直带着 “date” (去程用入住日期、回程用离店日期), 但 `server.py` 的 `apply_selection()` 的 `flight_picker` 分支手动拼字典存进 `trip_plan.add_flight()` 时漏了这一项, 候选有、存进去的没了; 就算存了, 前端 `web/index.html` 的机票卡片渲染也压根没有引用 `f.date`。已解决: 两处都补上——`server.py` 存的时候带上 “date”: `item.get("date")`, 前端卡片的 `.sub` 那行加上日期显示。 `trip_plan.add_flight()` 的字段说明文档字符串也顺手更新 (之前没提这个字段)。
7. **订酒店地图上出现了异地酒店** (2026-09-17, 用户真实预订时反馈) : `ota_hotel_agent._search_real_hotels()` 的锚点合理性校验 (第19条加的, 挡“中西药局旧址被匹配到美国圣路易斯”这类跑偏) 有个真实 bug——`on_target` (离锚点 `_ANCHOR_SANITY_KM` 内的候选) 算出来只用来判断“要不要整批放弃重查”, 判断完之后却没拿这个过滤结果替换 `hotels`: 5 个候选里只要有 1 个在范围内就不会触发重查, 但混进最终候选的还是没过滤的原始 5 个, 跑偏的酒店照样会跟着正常的一起出现在地图上。另外, 没有 `anchor` 的城市级搜索 (比如 `hotel_preference=“度假酒店”` 场景) 之前压根没有任何坐标合理性校验, `RollingGo` 城市级匹配理论上也可能出问题, 只是没人踩到过。已解决:

- `on_target` 非空时真的用它替换 `hotels` (`hotels = on_target`)，不是算完就扔，跑偏的候选不会再混进最终结果。
- 新增 `_city_center(city)`：用 `map_tool.geocode(city)` (真实地理编码，城市名是标准行政区划名字，不是生僻地标，可靠) 查城市中心点，查不到就返回 `None`，调用方跳过校验不强求。
- 新增 `_CITY_SANITY_KM=80` 兜底校验：不管走没走锚点，最后都会拿结果跟城市中心比一次距离——这层校验跟锚点校验管的不是同一件事（锚点校验管“离行程地点近不近”，这个管“压根有没有查到别的城市/国家去”），哪怕锚点校验已经通过，也不代表整个查询没有串到别的城市。

真实验证：模拟“5个候选混着1个异地酒店”的场景，锚点校验/城市级校验都能正确把异地酒店过滤掉，只剩本地候选；

`_city_center()` 查不到时优雅跳过校验，不报错、不误杀。

8. **酒店/机票偏好问过一次就全程不再问，但用户指出这是“这一趟去哪儿”的定制，不是绑在人身的人格变量** (2026-09-17, 用户提出“去福州可能更想吃，去香港可能更想打卡，去岷港可能想度假，每次都不同”)：第26条加的 `trip_preferences` 门槛只看 `prefs.is_complete()`，问完（或者用户“继续”用默认值兜底）之后这个 session 里不管后面请求哪个城市都不会再问——隐含假设是“这两个偏好是跟人绑定的”，但用户指出这两个字段其实是跟这一趟行程“绑定的”，换目的地就该重新问。**已解决**：  
`new_shared_state()` 新增 `trip_preferences_city` 字段，记录当前 `trip_preferences` 是为哪个城市答的；`orchestrate()` 里这次请求提取到的城市 (`_extract_city()`，跟 `content_agent.py` 独立一份同样的子串匹配思路) 如果跟这个字段不一样，就把 `prefs` 整个重置成空白（两个字段都重新问，不是只问少的那部分——上次答案是为另一个城市答的，沿用没有意义），重新触发确认门槛。提取不到城市（这句话没提、`shared_state["city"]` 也没有）就不重置，避免无谓打断。追问期间暂存的 `pending_trip_request` 旁边新增 `pending_trip_city`，等用户答完才真正写进 `trip_preferences_city`，没答完之前不会污染这个记录。

真实验证：真实两轮对话，先在“澳门”问完确认答成“度假酒店/省钱”；第二轮请求“香港”的行程，确认门槛正确重新触发（不是直接沿用“度假酒店/省钱”），`trip_preferences` 被重置成空白。

9. **新增：机票酒店行程都定完之后调 `exception_agent` 查真实天气，标出行程里可能下雨/下雪的天，雨大就问要不要调** (2026-09-17, 用户提出)：  
`exception_agent.py` 之前只有 `run()` (子串匹配“这句话有没有提到行程里的地点”) 和 `check_weather()` (查某个城市当前有没有真实生效的灾害预警)，都不是“给已经排定的每一天分别查天气、标进行程”这种颗粒度。新增：
- `_day_dates_from_flight(shared_state)`：`trip_plan.days` 的 key (“day-1” / “day-2” ...) 是 `route_agent.schedule()` 排班时按顺序分配的占位符，不是真实日历日期（已知的设计取舍）——借去程航班的真实 `date` 字段（第32条刚补上的）倒推，`day-1` 是航班抵达那天，`day-N` 顺推 `N-1` 天。这也是为什么这个功能必须等机票订完才能查：没有真实日期，没法把 `weather_tool.get_hourly_forecast()` 查回来的真实日历天气跟“day-N”这种占位符对上。
  - `check_itinerary_weather(shared_state)`：对 `trip_plan.days` 里每一天（按 `_day_dates_from_flight()` 换算出真实日期后）查一次天气预报，按这天最后一个有坐标的节点查（跟 `route_agent.schedule()` 排完一天查天气用的是同一个“查最后一个点”思路，不用每站都查）。降雨概率或者有降雪的天，写一条进 `trip_plan.weather_alerts`（前端本来就在渲染这个字段，不用改前端）；降雨概率达到阈值（跟 `route_agent.schedule()` 的降雨提醒用的是同一个 0.6）就整体标 `needs_replan=True`，返回一句“要不要调整”的问句——这是**纯信息性标记**（不删除/修改任何行程节点），不受这个模块“提案制、不自动执行”限制里“会真的改变行程结构的操作”那条规则约束；真要怎么调整还是得用户自己在对话里说清楚，不猜、不自动执行。
  - `server.py` 新增 `_trip_fully_booked()` (机票+酒店+每日行程是不是都有了)，`POST /widget-response` 确认完（不管是哪个 widget）之后检查这个条件，刚好凑齐就调一次 `check_itinerary_weather()`，天气提醒并衔接回复里：  
`state["weather_checked"]` 记一次就不再重复查（不然每多订一次东西——比如换酒店——都要重查一遍，真实 API 调用没必要）。

真实验证：真实调用和风天气 API 查当天澳门天气（降雨概率 16%），正确标成 `severity="minor"`、`needs_replan=False`；模拟高降雨概率场景验证 `needs_replan=True` 且正确生成“要不要调整”的问句，`weather_alerts` 里写入对应记录。

10. **订票订房不管用户说哪天出发，永远查“明天起2晚” + 自由组句回复编节日日期** (2026-09-17, 团队成员真实测试反馈)：

- `orchestrate()` 调 `ota_hotel_agent.run()` 的地方一直没传 `date_range` 参数 (`ota_hotel_agent.run(shared_state, location=None, user_message=user_message)`) —— `date_range=None` 时 `_parse_date_range()` 直接退回默认值：  
`check_in` 是“明天”，住 2 晚，**跟用户说了什么日期完全无关**。真实测试时系统日期是 9/14，“明天”正好是 9/15，不管用户说 9/24-9/27 还是别的，查出来的永远是 9/15-9/17，不是巧合，是压根没人把用户说的日期解析出来传下去。**已解决**：新增 `_extract_date_range()` (跟 `content_agent._DATE_RANGE_PATTERN` 同样的正则思路，独立一份，这边要的是真实公历日期不是天数)，`orchestrate()` 命中 `booking` 意图时解析当前消息里的日期区间传给 `ota_hotel_agent.run()`；没有年份信息，日期已经过了今年就推到明年（没人会订“去年”的机票）；解析不出来就是 `None`，退回原来的默认值，不强求。

- 自由组句回复（`orchestrate()`）里没命中 `restaurant / cancel` 时走的 LLM 组句分支）被问到“2026年中秋节是哪天”，编出了“9/17”——真实日期（`zhdate` 库算出来的）是 **9/25**，差了整整 8 天。`llm_tool.call_llm()` 是纯聊天补全调用，没有联网搜索也没有工具调用能力，农历转公历这种事对 LLM 来说本来就很容易凭训练数据编错。**已解决**：新增 `holiday_tool.py`（本地农历库 `zhdate` 算农历固定节日，公历固定节日直接算，不需要联网/API key，比接一个真实的联网搜索工具轻量得多——这类“哪天是哪个节日”的问题本身就是可以精确计算的事实，不是需要搜索的信息），收录春节/元宵/端午/七夕/中秋/重阳/腊八（农历）+ 元旦/劳动节/国庆节（公历）。`orchestrator_agent._resolve_holiday_mentions()` 检查用户这句话有没有提到认识的节日名，查出真实日期后跟着 `results / trip_plan / persona` 一起塞进给 LLM 的 `context`（新增 `holiday_facts` 字段），`system prompt` 明确要求“用户提到的节日只能按 `holiday_facts` 给的日期回答，没有的节日不要自己编，说不确定”。日期如果已经是今年过去的日子，自动换算成明年的（同样的“没人会问去年的节日”逻辑）。

真实验证：`_extract_date_range("9/24-9/27想去澳门")` 正确解析出 "2026-09-24~2026-09-27"；  
`holiday_tool.get_holiday_date("中秋节", 2026)` 真实算出 "2026-09-25"，`_resolve_holiday_mentions()` 正确识别节日提及并换算过期年份（问“春节”，2026年的已经过了，正确给出2027年的真实日期）。91 个测试全过。

- 删除 `restaurant_agent`（餐厅演示技能）——不接真实数据，还会把整段对话卡死在死胡同里**（2026-09-17，用户拿到团队成员真实测试的聊天截图，发现对方问“那按9/15-9/17的3天2晚来规划”这种纯粹推进日期确认的话，被分类成了 `restaurant` 意图，之后整段对话再也没跳出来——一路生成越来越像真的“3天2晚参考草案”（带具体航班号/酒店名/演示餐厅），但 `trip_plan.days` 全程是空的，左侧“还没有行程安排”贯穿了 4 张截图）：

- 查实根因有两层**：① `restaurant_agent.py`（`fellow` 分支早期写的）查的是 `restaurant_tools.py` 里硬编码的 3 条“演示餐厅 A/B/C”，从来没用过 `store.py` 的真实社区帖子库——跟这个项目现在“真实数据不瞎编”的其它部分（真实天气/真实酒店/真实路线/真实候选）完全不是一回事，纯属早期占位从没升级；② `_INTENT_SYSTEM_PROMPT` 里 `restaurant` 那条有一句“包括前文餐饮需求的修改、追问与回忆”，本意是让用户能连续调整餐厅筛选条件不用每次重新说“推荐餐厅”，但太宽泛——只要历史里出现过餐饮相关内容，LLM 就容易把后续任何模糊的推进类消息也当成“餐饮需求的追问”粘滞进去；`orchestrate()` 里 `elif "content" in intents and "restaurant" not in intents`：这行还会让 `restaurant` 一旦命中就直接跳过 `content_agent`，哪怕两个意图同时命中也不例外。
- 用户确认**：既然这个技能根本不用真实数据库，直接删掉，不用修。**已解决**：整个删除 `orchestrator/agents/restaurant_agent.py`、`orchestrator/restaurant_tools.py`；`orchestrator_agent.py` 不再 `import`/调度它，`_INTENT_SYSTEM_PROMPT` 去掉 `restaurant` 这一条（`content` 的说明顺带改成“景点或美食推荐”，明确美食推荐现在完全由 `content_agent` 的真实数据覆盖）；`classify_intent()` 的白名单也去掉 `restaurant` 这个 key——就算 LLM 因为训练偏置还是吐出这个词，也会被过滤掉，不会走到任何特殊分支，从根上让这类“卡在死胡同”的情况不可能再发生。回复组句、`shared_state["messages"]` 写入逻辑也一并简化（去掉之前专门为 `restaurant_agent` 自己管理对话历史留的特殊分支）。
- 顺势收紧了自由组句回复的系统提示词**：明确禁止自己拼一份带具体航班号/酒店名/逐天时间表的完整行程草案来冒充规划（哪怕标注“未验证提案”也不行）——那份看起来越来越像真的“参考草案”就是这么来的，即使技能本身删了，自由组句分支本来就有能力编出类似的东西，这条限制是独立于 `restaurant_agent` 删除之外的第二层防护。

真实验证：`classify_intent()` 即使被 `mock` 成返回 `["restaurant", "content"]`，白名单过滤后也只剩 `["content"]`；确认 `orchestrator_agent` 模块上已经没有 `restaurant_agent` 这个属性。83 个测试全过（删除了 8 个纯 `restaurant_agent / restaurant_tools` 相关的测试）。

- 新增：多城市行程（“前两天去澳门，后两天去香港”，或者先规划完一个城市后面接着说想去另一个城市）**（2026-09-17，用户提出）：查实每一层都假设整趟行程只有一个城市——`_extract_city()`（`content_agent.py / orchestrator_agent.py` 各一份）在（“澳门”，“香港”）里挨个找，命中第一个就返回，两个城市都提到时后一个直接被忽略；`shared_state["city"]` 只在会话创建时设一次（`server.py` 的 `_DEMO_CITY`），**整个对话过程中从没被更新过**，`apply_selection()` 排班时用的就是这个过期的静态值，哪怕候选实际是另一个城市查的。**已解决**，两种触发方式共用同一套机制：

- `shared_state["city"]` 改成跟着最新一句话动态更新：`orchestrate()` 里这次请求提取到城市就写回 `shared_state["city"]`，提取不到才保留原值。
- 新增 `last_content_city`（跟已有的 `last_content_candidates / last_trip_day_count` 放一起）：记住这批候选是给哪个城市查的，`attraction_picker` 确认时排班优先用这个，不用可能已经被后续对话覆盖掉的 `state["city"]`。
- 一句话清楚的多城市**（“前两天去澳门，后两天去香港”）：新增 `_parse_multi_city_segments()`，跟 `_parse_nearby_request() / _parse_preference_answer()` 一样的思路——自然语言里“前两天”/“先...再...”这种表达方式太多，正则覆盖不过来，交给一次 `MODEL_LIGHT` 调用解析成 `[{"city": "澳门", "days": 2}, {"city": "香港", "days": 2}]` 有序分段，只解析出 1 个城市就不算“多城市”，退回单城市流程。命中多城市时，`orchestrate()` 只处理第一段（用 `f"{city}玩{days}天”` 拼一个合成的

`location_hint` 喂给 `content_agent.run()`，复用它已有的 `_extract_day_count()` 解析，不用改 `content_agent.py` 的接口)，剩下的段存进新增的 `pending_city_segments`。

- **续接机制**（一句话多城市 / 增量式“接着还想去另一个城市”是同一套）：`server.py` 的 `apply_selection()` 排完当前这段确认的候选之后，检查 `pending_city_segments` 有没有排队等着的下一段——有就自动调 `content_agent.run()` 查下一个城市的候选、生成新的 `attraction_picker / post_list` 卡片塞进 `pending_widgets`，衔接回复里提示“接下来是XX的部分”，不用用户重新开口。`widget_response()` 路由里“摘掉这次确认掉的卡片”那行代码挪到了 `apply_selection()` 调用之前（不然新塞进去的下一段卡片会被这行代码当成“这次确认掉的”一起摘掉）。
- **顺带修的一个连带问题**：`state["weather_checked"]` 原来是“整个 session 只查一次”的布尔值，多城市行程分批订机票时，第一个城市订完触发一次检查后，第二个城市新增的机票永远不会再被检查到。改成 `weather_checked_flight_count`（记录上次检查时 `trip_plan.flights` 有几条），机票数量比上次检查时更多才重新查，不是布尔开关。

真实验证：真实调用 `_parse_multi_city_segments("我前两天去澳门，后两天去香港，一起帮我规划")`，正确解析出 `[{"city": "澳门", "days": 2}, {"city": "香港", "days": 2}]`；合成的 `location_hint`（“澳门玩2天”）真实过一遍 `_extract_day_count()` 正确得到 2。88 个测试全过。